Issue Trees with MECE Studio

A practitioner’s guide to structured problem solving

Dann Bleeker Pedersen

Contents

BEGINNING

Foreword

STRUCTURING THE PROBLEM

Start with the question
The issue tree
MECE: no overlaps, no gaps
Ways to decompose

WORKING THE TREE

Working hypotheses
Prioritise the 80/20
Evidence
Doing the numbers

COMMUNICATING THE ANSWER

Answer-first
A worked example
Presenting and sharing

APPENDICES

Appendix A — Glossary
Appendix B — Keyboard reference
Appendix C — Further reading
Appendix D — Working in MECE Studio

Foreword

Every hard problem starts the same way: someone walks into a room, lists what they know, and then — before anyone has agreed on what the actual question is — someone else proposes a solution. The room fractures. Half the people argue about why the solution won't work. The other half defend it. An hour passes. Nothing is resolved.

This is not a failure of intelligence. The room is usually full of smart, motivated people. It is a failure of structure. And structure is something you can learn.

How smart people fail at hard problems

There are four traps that catch almost everyone, regardless of experience.

The favourite-solution trap. The moment a problem is described, our brains start pattern-matching to past situations and generating candidate answers. That's useful — experience matters — but it becomes a trap when the candidate answer hardens into a conclusion before any analysis has happened. The fix feels obvious, the investigation is already mentally over, and any evidence that doesn't fit gets unconsciously downweighted. Smart people are, if anything, more vulnerable to this: they're faster at generating plausible-sounding solutions and better at arguing for them.

The boiling-the-ocean trap. The opposite failure. Faced with uncertainty, some teams decide to analyse everything. Gather all the data. Interview all the stakeholders. Map all the processes. Six weeks later they have a lot of findings, no clear story, and a deadline that passed while they were still collecting. The energy was real; the focus wasn't.

The anchoring trap. The first explanation you hear for a problem tends to stick, even when better explanations emerge later. A CFO says "it must be the new competitor" in the opening meeting, and the team unconsciously frames every piece of evidence around that hypothesis. They're not lying or lazy — anchoring is a deeply human cognitive quirk. But it means whole branches of the real explanation get left unexplored.

The invisible-option trap. Perhaps the most dangerous of all. You can have a rigorous debate between Option A and Option B, run all the numbers correctly, and still miss the right answer completely — because Option C was never put on the table. Options that aren't generated can't be evaluated. And options that aren't generated usually aren't missing by accident: they're the ones that are uncomfortable, unconventional, or just too far outside the team's natural frame of reference.

The common thread across all four traps is the same: the team jumped to the middle of the problem without a shared map of what the problem actually looks like.

What a shared map buys you

An **issue tree** is that shared map. It is a question broken into smaller questions, and those smaller questions broken further, until the pieces are small enough to investigate directly. It sounds simple — it is simple — but the consequences are substantial.

No double-counting. When you decompose a problem cleanly, each sub-issue lives in exactly one place. If "falling revenue" and "falling volume" are both on your list, you're probably counting the same thing twice

and inflating its apparent importance. A well-built tree makes that visible immediately.

No blind spots. The discipline of asking "is this list complete?" forces you to look for what's missing, not just explain what's present. Entire categories of explanation that would otherwise go unexamined become visible as gaps — white space on the map that still needs to be filled.

Divide the work cleanly. Once you have a map, you can assign branches. Two analysts can work on the revenue side and the costs side simultaneously without stepping on each other, because the boundary between them is explicit. The work is parallelisable in a way that unstructured investigation never is.

A shared language. When the team reconvenes, everyone is oriented to the same structure. "I looked at the volume driver on the revenue side — here's what I found" is a statement that locates itself on the map. Nobody has to spend ten minutes figuring out how a finding relates to the overall question.

A forcing function against favourite solutions. If your tree has four branches and you've only investigated one, the incompleteness is visible. The structure creates accountability — not to a person, but to the logic.

> *A good issue tree doesn't just organize what you know. It makes visible what you don't.*

The MECE principle

Alongside the idea of an issue tree, this book is built around one quality test: **MECE**, which stands for **Mutually Exclusive, Collectively Exhaustive**. A split is MECE if its children don't overlap (mutually exclusive) and together cover the entire parent question without leaving anything out (collectively exhaustive).

MECE is a standard from the consulting world, but it belongs to everyone. It is really just a formal name for the two most common structural mistakes: saying the same thing twice, and missing something important. You'll spend a full chapter getting comfortable with it, but the core intuition is quick to develop — and once you have it, you'll notice MECE violations everywhere, in strategy decks, in board presentations, in your own first drafts.

What this book is

This is a practitioner's guide. It is not a textbook. It does not assume any background in consulting, strategy, or formal logic. It assumes only that you face hard, messy problems at work and want better tools for structuring your thinking.

Each chapter teaches one concept and pairs it with hands-on practice in **MECE Studio** — a free, local-first app that runs in your browser and stores everything on your device. You don't need an account. You don't need to pay anything. The app is designed to be a thinking partner: it handles the layout and the bookkeeping so you can focus on the ideas.

That said, this is a book about method, not software. If you already use a whiteboard, a mind-mapping tool, or a spreadsheet for structured analysis, the concepts here apply equally. MECE Studio is the companion, not the subject.

Who this is for

Analysts who need to structure an investigation before they know what they're looking for, and who want a clean way to present findings when they're done.

Managers who facilitate problem-solving with their teams and want a way to stop conversations from cycling in circles.

Founders and operators who make high-stakes decisions under uncertainty and can't afford to miss an important option or mistake the cause.

Consultants — junior and experienced alike — for whom structured problem solving is a core professional skill, and who want a tool that keeps up with how fast the thinking actually moves.

You don't need to read this front to back in one sitting. Each chapter is self-contained enough to revisit when you hit a specific obstacle. That said, the concepts do build on each other, and the first read is best taken in order.

How to read it

The book follows the shape of a real analysis. You start by pinning down the right question — the chapter on **Start with the question** explains why the root of your tree matters so much, and how to sharpen a vague concern into something you can actually decompose. Then you build the tree itself, using the decomposition types covered in the chapter on **Ways to decompose**. You test each split for MECE, form and update working hypotheses, decide where to focus using the 80/20 lens, bring in evidence, and — when the numbers matter — you quantify.

At the end, you synthesise: distill everything you've learned into a clear, answer-first communication. The chapter on **Answer-first** covers how to do this without losing the rigour underneath.

A full worked example then walks one problem from blank canvas to finished synthesis, so you can see all the pieces fit together. A closing chapter covers getting the tree in front of an audience — presenting it live, printing it, and exporting it in whatever form the room needs.

There is a running example threaded through the whole book — a business whose profits are falling, decomposed into a revenue branch and a costs branch, each with its own drivers. It's simple enough to hold in your head, complex enough to illustrate the interesting cases. You'll recognise it quickly once it appears, and you'll see the same tree grow and evolve as each new concept is introduced.

Structured problem solving is a learnable skill. Not everyone who practises it becomes a great strategist, but almost everyone who practises it becomes a clearer thinker — someone who asks better questions, builds more honest arguments, and catches their own blind spots before someone else does.

That's a worthwhile return on a few hours of reading.

Let's get started.

Start with the question

Before you draw a single node, before you think about decomposition, before you ask what the data says — you need to know what question you are actually trying to answer.

This sounds obvious. It almost never gets done properly.

The **key question** sits at the root of your issue tree. Every branch, every sub-issue, every hypothesis you form, every piece of evidence you gather — all of it exists to answer that one question. If the question is wrong, the entire tree is wrong. You can do brilliant analysis in perfectly structured branches and still produce a useless result, because you answered the wrong thing.

Getting the key question right is the highest-leverage move in the whole process. It is also where most teams spend the least time.

What makes a question good

A good key question has three properties.

It is specific. "What's wrong with the business?" is not a question — it's an anxiety. A specific question names the phenomenon you're investigating, scopes it in time or context, and points toward a real decision. "Why have profits fallen by 18% over the past two quarters?" is specific. You know what you're investigating (profitability), you know the magnitude (18%), and you know the window (two quarters). That specificity shapes the tree before you've written a single branch.

It is decision-relevant. A good question exists because someone needs to act on the answer. If no one is going to make a different decision based on what you find, the question isn't worth asking — at least not right now. "Why are profits falling?" matters because the leadership team needs to decide what to fix. "What caused profits to fall in Q3 2019?" might be historically interesting but isn't decision-relevant unless there's still something to do about it.

It is answerable. This rules out questions that are genuinely unanswerable within your constraints, but also questions that are so broad they can never be satisfactorily closed. "How do we become the market leader in five years?" is not a question you can answer with an issue tree in a three-week analysis. You might be able to answer "What are the two or three strategic moves with the highest potential impact on our market position in the next 18 months?" — and that's a much more useful starting point.

One more criterion that often gets overlooked: a good key question is phrased as an **actual question**. Not a topic ("Profitability analysis"), not a statement ("We need to improve margins"), but a sentence that ends in a question mark and demands a specific kind of answer. The phrasing matters because it disciplines you. "Why are profits falling?" requires you to identify causes. "Should we enter the Asian market?" requires you to reach a yes/no recommendation. Different phrasings pull the tree in very different directions.

The Situation–Complication–Question frame

One of the most practical tools for arriving at a good key question is the **Situation–Complication–Question** (SCQ) frame — a three-step narrative structure borrowed from the work of Barbara Minto.

Situation is the stable context: what everyone agrees is true, the baseline facts. It is not the problem — it's the setup. "We are a mid-sized consumer goods company with a strong brand in three core categories. Our profitability has historically tracked at around 12% net margin."

Complication is what has changed, or what is under threat. Something disrupted the stable situation and created the need for analysis. "Over the last two quarters, our net margin has dropped to 9.4%. We don't yet understand why."

Question follows directly from the complication. It is the sharpest, most decision-relevant formulation of what you now need to know. "Why have profits fallen, and what should we do to restore them?"

The SCQ frame is useful for two reasons. First, it forces you to separate what is stable background from what is actually the problem — a distinction that is surprisingly easy to blur. Second, it makes the question feel inevitable: once you've stated the situation and the complication clearly, the question almost writes itself. If the question feels forced or artificial after you've written the SCQ, that's a signal the complication isn't quite right yet.

Try it whenever you're unsure whether your key question is the right one. Write out the situation in two or three sentences. Write the complication in one or two. Then write the question. Read all three together. Does the question follow? Is it the sharpest version of what the complication demands to know?

Failure modes

Knowing what makes a question good is easier if you've seen the common ways questions go wrong.

Too vague. "What's wrong with the business?" "How can we do better?" "What are our strategic options?" These questions open into everything and close into nothing. They don't shape a tree — they produce a brainstorm. Any issue tree built on a vague question becomes a list of every potentially relevant topic, which is not an issue tree; it's a table of contents for a textbook.

Leading or loaded. "Why is our competitor's strategy a threat to us?" assumes the competitor's strategy is a threat. That assumption might be right — but if it's baked into the question, your tree will never be able to find out that it's wrong. A loaded question is an anchor disguised as a question. It seems specific (it is about a specific thing) but it pre-determines the answer before any analysis has happened.

Unanswerable. "What is the right price for our product?" sounds specific, but without constraints on the type of analysis, the data available, and the decision being made, it has no bottom. Similarly, "What should our five-year strategy be?" is not something a three-branch tree can answer. Unanswerable questions tend to produce exhausting projects that eventually collapse into a set of recommendations that everyone already agreed with before the analysis started.

Multiple questions in one. "Why are profits falling, and should we acquire a competitor, and how do we think about our international expansion?" This is three questions. They might all matter to the same team at the same time, but bundling them into one key question means your tree will have branches that don't logically connect at the root. Separate them. Address them with separate trees if necessary. Mixing questions at the root is one of the most reliable ways to produce an issue tree that feels right but can't actually be answered.

Sharpening the question reshapes the tree

The single most powerful demonstration of why the key question matters is to watch what happens to the tree when you sharpen it.

Suppose your starting point is: "What's wrong with the business?"

That's not a question — it's a shrug. An issue tree built on it might look like: products, customers, operations, finance, people, technology. You've just listed business functions. Nothing is excluded, nothing is specifically implicated, and there's no way to know when you've answered it.

Now apply the SCQ frame. The situation: stable mid-sized consumer goods company, historically 12% net margin. The complication: margin has dropped to 9.4% over two quarters, cause unknown. The question: "Why have profits fallen, and what can we do about it?"

That question now has real shape. Profits are either a revenue problem or a costs problem (or both). The revenue side breaks into volume and price. The costs side breaks into input costs and efficiency. Immediately you have a structure — two top-level branches, each decomposable into drivers — and you know what you're looking for: the quantitative contribution of each driver to the 2.6-point margin drop.

Compare the two trees. The first has six vague branches that nobody knows how to investigate. The second has a clear logic: answer the small questions (what happened to volume? to price? to input costs? to efficiency?) and you answer the big one (why did profits fall?). That logic — where answering the children answers the parent — is exactly the property that makes an issue tree useful. And it flows directly from the quality of the key question.

That's not a hypothetical. It is what actually happens in practice: teams who spend an extra thirty minutes sharpening the question before building the tree save days of misdirected analysis later.

> *The question is the tree. Get it wrong, and no amount of careful decomposition will save you.*

Sharpening further: splitting a question or narrowing it

Sometimes the right move isn't just to phrase the question better — it's to scope it more tightly.

"Why have profits fallen, and what can we do about it?" is already a strong question, but it contains two distinct sub-questions: a diagnostic one (why?) and a prescriptive one (what to do?). Depending on your situation, you might want to run a separate tree for each, or you might want to acknowledge that the second question depends on fully answering the first and structure the analysis in two phases.

Alternatively, if you know that the revenue side is already well-understood — perhaps there was a well-documented volume decline due to a product recall — you might narrow the question to "Why have costs risen relative to revenue, and what are the highest-leverage interventions?" That narrowing produces a sharper, faster analysis.

Neither choice is universally right. The point is that the key question is a design decision, not a given. You are choosing what kind of analysis to run, what decision it needs to support, and what constraints on time and data it has to respect. That design decision is most productively made explicitly, at the start, before the tree is built — not implicitly, mid-analysis, when it's expensive to change.

Why trees and how trees

The diagnostic / prescriptive split in the last section is worth making explicit, because it decides the *shape* of everything below the root. A **why tree** is diagnostic: it breaks a problem into its possible causes ("Why have profits fallen?" → revenue causes, cost causes). A **how tree** is prescriptive: it lays out and compares the ways to act ("How do we restore margin?" → raise price, cut input costs, shift the mix). The two are not interchangeable, and a single tree should ask *one* of them, not both. Mixing "why is this happening?" with "what should we do about it?" at the same level produces branches that don't belong together — the same root failure as bundling multiple questions.

Arnaud Chevallier, who has written extensively on structured problem solving, states the rule plainly: a tree runs in one direction. Keep the diagnosis and the prescription in separate trees (or separate phases of the work), and each stays MECE and answerable on its own terms. Often the how tree only makes sense *after* the why tree has found the cause — you diagnose first, then decide.

In MECE Studio you can tag a tree **why** or **how** from the Problem brief (below), and a badge in the header shows which. Once a tree is tagged, the app's coaching keeps it honest: it flags a branch that opens with the opposite question word, and a step-by-step *process* split inside a *how* tree — because a how tree is meant to weigh alternatives, not lay out a sequence of steps. It is a nudge, never a hard error; the direction of the tree is yours to set.

Putting it into practice

In MECE Studio, the root node holds your key question. It's the first thing you see when you open a new document, and it sits at the left edge of the canvas, with every branch growing rightward from it. Renaming the root node to your sharpened key question is naturally the first move you make — and it should feel like a commitment. You're not just labelling a box; you're declaring what the whole tree is for.

If you find yourself frequently rewriting the root question mid-session, that's useful feedback: the question wasn't settled yet. Pause, apply the SCQ frame, and re-anchor before adding more branches. A tree built on a shifting question grows in all directions and satisfies none of them.

MECE Studio gives that framing a permanent home. The **Problem brief** button in the Answer banner opens a one-page **Problem Identity Card** — *Situation*, *Complication*, *Owner*, *Decision-makers*, *Success criteria*, *In scope*, *Out of scope*, and *Desired outcome*. It is the SCQ frame widened into the whole framing of the engagement: not just the situation and complication, but who owns the decision, who has to be convinced, what "solved" actually looks like, and what you are deliberately choosing *not* to tackle. Every field is optional and saves as you type. The Situation and Complication then lead the answer-first synthesis (see *Answer-first*) as a Situation → Complication → Resolution intro, so the framing you do here is reused when you communicate the answer. The card also nudges you when the key question isn't sharp — when it isn't phrased as a question, bundles two questions, or runs too long, the exact failure modes from earlier in this chapter.

When you're ready, take a real problem you're working on — something with genuine stakes, not a toy example — and write out the SCQ for it before you open the app. Write the situation in two sentences. Write the complication in one. Then write the question. Make it specific, decision-relevant, and answerable. Make it a real question.

That is your root node. The rest of the tree follows from there — and that's exactly where the next chapter picks up.

The issue tree

An issue tree is a question broken into smaller questions, and those smaller questions broken further still, until every piece is small enough to investigate directly. That's the whole definition. The power isn't in the metaphor — it's in what the structure forces you to do.

Once you have a well-formed key question at the root (as covered in the chapter on Start with the question), your job is to decompose it. What must you answer in order to answer the root question? Those answers become your second level. What must you answer in order to answer each of those? Those become your third level. You keep going until you reach questions that are concrete enough to take to data, to an expert, or to a calculation.

The result is a tree: one root, branches spreading outward, with leaves at the tips where the investigation actually happens.

The parent–child logic

The governing logic of an issue tree is simple and strict: **to answer a parent question, you must answer all of its children.** This is what makes an issue tree different from a topic list or a brainstorm.

If your root question is "Why have profits fallen?" and your children are "What happened to revenue?" and "What happened to costs?", then the claim is: if you fully answer both children, you will have fully answered the root. Revenue and costs between them account for all of profit — so yes, if you understand what happened on both sides, you understand what happened to profit. The parent–child logic holds.

Now suppose you add a third child: "What is our competitive position?" That might be an interesting question, but it doesn't have the same logical relationship to the root. Your competitive position might *explain* what happened to revenue or costs, but it isn't a *component* of the profit question in the same structural sense. It belongs deeper in the tree — as a potential driver under the revenue or costs branch — not at the top level.

This distinction is the most important one in building a good tree. There are always many things worth investigating. The question is where they belong in the structure. Sub-issues should be decompositions of their parent — components, drivers, phases, segments — not just associated topics that seem related.

> *Every child must be necessary for the parent, and together the children must be sufficient. That test catches most structural errors.*

Reading a tree

Issue trees grow left to right in MECE Studio, which matches how most people read: from the question on the left to the answers on the right. Each node is a question or sub-issue. Lines connect parent to children. The further right you go, the more specific and investigable the questions become.

A small issue tree looks like this:

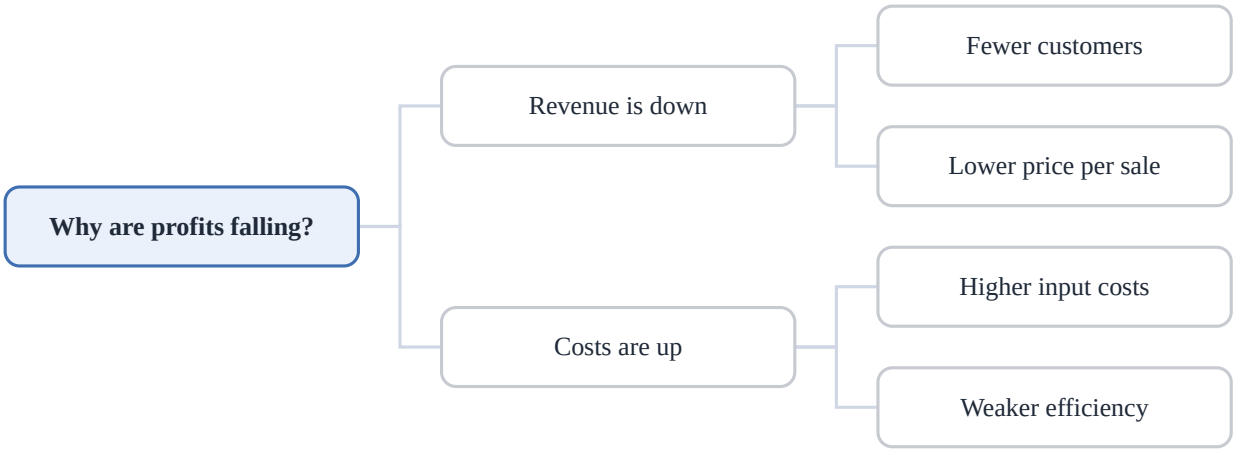


Figure 1. An issue tree decomposes a question into sub-issues. Profit splits into revenue and cost; each splits again into its drivers.

In Figure 1, the root question — "Why have profits fallen?" — splits into two branches: revenue and costs. The revenue branch splits further into volume (fewer customers, or fewer purchases per customer) and price (lower prices realised, or a less favourable product mix). The costs branch splits into input costs (raw materials, labour, logistics) and efficiency (output per unit of input). At the leaf level, you have questions that a competent analyst can take to data: "Has average transaction value declined?" "What has happened to raw material costs per unit?"

That is the complete anatomy. Root question → mid-level drivers → leaf-level investigable questions. Most useful trees are three to four levels deep; very few need to go beyond five.

How to grow a tree

You grow a tree by asking one question repeatedly: "To answer this, what do I need to know?"

Start at the root. Ask the question. Write down what you'd need to know to answer it — not everything that's potentially relevant, but the specific sub-questions whose answers would constitute an answer to the parent. Those are your first-level children.

Then take each child and ask the same question again. What would you need to know to answer this? Keep going until you reach questions that feel directly investigable — something you could assign to an analyst with a clear brief, or answer yourself with an afternoon in the data.

In MECE Studio, you add a child to any node directly on the canvas. The app offers decomposition types — segment, process, binary, formula, framework, freeform — each of which seeds type-appropriate starter children as a scaffold. These are starting points, not prescriptions; you'll rename and adjust them as you develop your thinking. The layout rearranges automatically as the tree grows, so you don't have to manage spacing or positioning. You can focus on the structure itself. The chapter on Ways to decompose goes deeper on how to choose among those types.

A few practical habits when building:

- **Name nodes as questions.** "Revenue" is a topic. "What happened to revenue?" is a question. The question form makes the parent–child logic testable: does answering this question help answer its parent? The topic form makes it easy to paper over structural sloppiness.

- **Add one level at a time.** Resist the urge to deep-dive into one branch while leaving others undeveloped. A tree with three levels on the revenue side and zero on the costs side is a sign that your favourite-solution instinct has taken over. Stay balanced until the structure is roughly symmetrical, then go deep where the evidence warrants.
- **Use provisional names.** Early in the build, your node names will be rough. That's fine. The purpose of the first pass is to get the structure right, not to produce polished phrasing. You'll refine names as understanding develops.

Depth versus breadth

There's a permanent tension in issue trees between going wider (more branches at the same level) and going deeper (more levels in a single branch). Neither is always right.

Too broad means your first level has six or eight branches, each vague enough that you can't tell which ones actually matter. The tree becomes hard to prioritize and harder to investigate, because the branches are too big to assign cleanly. If you find yourself with more than four or five first-level children, ask whether some of them should be grouped under a common parent that you haven't named yet.

Too deep too fast means you've dived into granular questions before establishing whether the high-level branch is even worth investigating. If "input costs" turn out to account for 0.1% of the profit decline, you don't need four levels of decomposition underneath them. Depth should be earned by the evidence, not preemptively assumed.

A useful rule of thumb: **go wide before you go deep**. Establish the full first level before developing any second level. Establish the full second level before developing any third. This keeps the tree balanced and ensures you don't overinvest in branches that don't matter.

The right depth for a branch is wherever you reach a question that's directly investigable. For some questions that might be level two; for others it might be level four. The tree's shape doesn't need to be symmetric — it needs to be honest about where the complexity actually lives.

When to stop

You stop adding children when the question is directly investigable: when you could write a brief for an analyst, run a query in your data tool, or make a quick calculation. That's a leaf node. It doesn't need children — it needs work.

You also stop when you've confirmed that a branch is not a meaningful contributor. If you investigate the price branch and find that average realised price has been flat for two years — no meaningful movement — then the price branch is closed. It's not deleted from the tree (it was a legitimate hypothesis that has now been answered), but it doesn't need to be developed further.

This is one of the underappreciated uses of an issue tree: it lets you close branches explicitly. An unstructured investigation can feel like it never quite reaches a conclusion because it's never clear when you've covered enough. An issue tree creates natural closure conditions. When every branch at the leaf level has been investigated and answered, the root question is answered. You know you're done.

The tree is a living document

Issue trees change as understanding develops. You'll add branches you didn't anticipate. You'll collapse branches that turned out to be dead ends. You'll occasionally restructure — realise that what you thought was a first-level issue is actually a driver of something else and move it down a level. That's not a sign of failure; it's a sign the analysis is working.

In MECE Studio you can drag nodes to reparent them, rename any node inline, and undo any change. The tree is meant to be edited iteratively, not built once and frozen. Keep a copy at a clean milestone if you want one (duplicate the tree, or save it to a file), but don't treat any version of the tree as permanent until the analysis is actually done.

Collapsing branches you've already investigated can help too — once you know that the costs side is fully explained by input costs and you've documented the finding, collapsing the costs branch lets you focus visual attention on the open revenue questions. The canvas is a working surface, not a final product.

A note on terminology

You'll sometimes hear issue trees called **logic trees**, **hypothesis trees**, or **value-driver trees**. They're closely related but not identical. A pure issue tree decomposes questions. A hypothesis tree makes an explicit claim at each node and tests it. A value-driver tree (covered in the chapter on doing the numbers) assigns numeric values to nodes and calculates upward through formula relationships.

MECE Studio supports all three modes — you can attach a hypothesis status to any node and run numeric values through formula splits — but the underlying structure is always a question broken into sub-questions. Start there, and the other modes layer on top naturally.

Connecting the structure to the method

The issue tree is the skeleton of the whole method. Everything else — checking MECE, forming hypotheses, prioritising where to investigate, gathering evidence, synthesising findings — happens in relation to the tree. When you find a gap, you add a branch. When you find evidence, you attach it to a node. When you conclude, you read from the leaves back to the root.

That's the architecture of the whole process: build the map, investigate the map, and report from the map. The next chapter takes up the MECE standard — the quality test that tells you whether your map has the right shape.

MECE: no overlaps, no gaps

An issue tree is only as good as its splits. You can have a beautifully branched diagram that still sends half your team chasing the same problem twice, or leaves an entire cause unexamined. The principle that prevents both failures is **MECE** — *Mutually Exclusive, Collectively Exhaustive* — and it is the discipline this book returns to again and again.

This chapter unpacks what MECE actually means, where it is judged, how to test it quickly, what common violations look like, and why the goal is useful-and-nearly-MECE rather than theoretical perfection.

What the two words mean

Mutually exclusive (ME) means no item belongs to two parts at the same time. Each sub-issue, each customer, each cost line lives in exactly one bucket. If the same thing can land in two places, you will count it twice, fight about ownership, and lose track of the total.

Collectively exhaustive (CE) means the parts, taken together, cover the whole. Nothing falls through the cracks. If your split of "why are costs high?" only covers labour and materials, and rent is the culprit, you will never find the answer — because your tree has no place for it.

> *MECE is not about elegance. It is about not losing anything and not counting anything twice.*

The two errors are different in character. An ME violation is usually visible and embarrassing — people argue about which bucket something belongs to. A CE violation is quieter and more dangerous — the answer is simply absent, and you may not notice until the project is over.

MECE is judged on a split, not the whole tree

This is the most important structural point in the chapter. A tree is a **stack of splits**, and MECE is checked one split at a time: one parent, its immediate children. You are not asking whether the entire tree is MECE; you are asking, for each parent node, whether its children are mutually exclusive with each other and collectively exhaustive of what the parent contains.

Take the classic "Why are profits falling?" tree. The root splits into **Revenue** and **Costs**. That is one split — is it MECE? Revenue and Costs do not overlap (a dollar is either income or an expense, not both), and together they account for all of profit (Profit = Revenue − Costs). Clean.

Now Revenue splits into **Price** and **Volume**. Another split, checked independently. Revenue = Price × Volume, so the arithmetic makes it exhaustive, and price and volume are distinct concepts. Also clean.

A violation at one level does not pollute other levels. A sloppy split of Costs does not affect whether the Revenue / Costs split is good. This is why you walk the tree split by split rather than judging it as a whole.

How to test a split

Two quick questions:

ME test — "Could something belong to two of these?" If you can think of one real example that fits two children, the split is not mutually exclusive. Stop and redesign it.

CE test — "Is there anything relevant that fits none of these?" If you can think of one real example that belongs to the parent but falls outside all the children, the split is not collectively exhaustive. Add a child or widen one that already exists.

Run both tests on each split before you move down the tree. It takes thirty seconds and saves hours.

Clean MECE splits

Some splits are almost always clean because the logic is built in.

Arithmetic. Profit = Revenue − Costs. If you can write the parent as an equation, the terms of the equation are MECE by construction — they are exhaustive because the equation is complete, and they are exclusive because each term is defined. This is why formula decomposition is so powerful. More in *Doing the numbers*.

Binary (A / not-A). Customers who churned / customers who did not churn. New product revenue / existing product revenue. Any complement pair is provably MECE: the two parts are exclusive by definition, and together they cover everything. When you are stuck on how to cut a question, a binary split is always safe and often clarifying.

Ordered stages with no overlap. Awareness → Consideration → Purchase → Retention. Each stage is defined by where the customer is in the journey; a customer is in exactly one stage at any point in time; and if the stages cover the full journey, the split is MECE.

Tempting non-MECE splits

Most violations follow a small number of patterns.

Mixed dimensions. "Customers: big / small / online." This mixes size (big, small) with channel (online). A large online customer belongs to three buckets. The dimensions are not aligned. The fix is to split on one dimension at a time: size first, then within each size group, channel.

Overlapping labels. "Cost drivers: labour / overhead / executive salaries." Executive salaries are overhead and labour. The labels overlap. When you find an overlap, either merge the overlapping categories or redefine them so each term is unique.

Vague catch-all that conceals a gap. "Revenue: domestic / international / other." If "other" is doing real work — hiding a meaningful category like licensing or government contracts — your tree will not prompt you to investigate it. An "Other" bucket is legitimate as a placeholder for genuinely miscellaneous items; it is a hiding place when it absorbs a category you know exists.

Gaps by omission. You split marketing spend into Digital and Events. What about print? PR? If you leave channels out, you will not investigate them. A CE gap is often invisible precisely because the missing item never appears in the tree.

The pragmatic tension

MECE is a standard to aim at, not a gate you must pass before doing any work. Three practical points:

A useful-and-nearly-MECE split beats a perfect one you can't act on. If the only way to make your customer split perfectly MECE requires nine dimensions and forty-two sub-categories, you have traded analytical clarity for analytical paralysis. A slightly impure split that separates the two or three things most likely to matter is better than a pristine one that is unusable.

The "Other" bucket is the honest escape hatch for exhaustiveness. Rather than leaving a gap, add a child called "Other / not elsewhere classified." This makes the split exhaustive, names the residual explicitly, and keeps it visible. The rule is that "Other" should be the smallest bucket at the end of the analysis — if Other turns out to be the largest driver, it needs to be broken open.

Document the compromise. When you knowingly accept a non-MECE split, note why. That transparency prevents a colleague from spending a week investigating an "overlap" you already knew about and chose to live with.

When a split is an argument, not a partition

Almost every split in this book is **inductive**: you take a whole and cut it into parts of the same kind — regions, cost lines, funnel stages — and MECE asks whether those parts overlap or leave a gap. That is what the ME and CE tests are for.

But not every decomposition is a partition. Some are **deductive** — an argument chain, where each child is a step that builds on the one before it toward a conclusion: *the market is growing → we have a right to win in it → therefore we should enter*. The children of a deductive split are not siblings carving up a space; they are premises in a line of reasoning. (Barbara Minto makes exactly this distinction: a group of ideas can relate in one of two ways — as a grouping or as an argument.) Asking whether "the market is growing" and "we have a right to win" are *mutually exclusive* is a category error — they are meant to depend on each other; that is the point.

So MECE simply does not apply to a deductive split. What you check instead is whether the chain *holds*: does each step follow from the one before, and does the conclusion follow from the steps together? A broken deductive chain is a hole in your logic, not an overlap in your buckets.

In MECE Studio you mark a split **deductive** on the inspector's **Logic** tab (inductive is the default, and most splits stay that way). The tool then stops overlap-checking that split — it will not flag its premises as colliding — and instead prompts you to confirm the chain is sound. Use it for the argument-shaped parts of a tree, and leave the rest inductive, where the MECE checks do their work.

How MECE Studio surfaces this live

MECE Studio embeds the MECE check into the canvas so you see problems as they arise rather than discovering them in a review meeting.

Every node that has children displays two small indicators — one for ME, one for CE. Each carries a glyph as well as a colour — ✓ pass, **!** needs review, **–** not checked — so the state reads in greyscale, for colour-blind readers, and to screen readers, not by colour alone. Select the node and open the inspector's **Logic** tab

to read a plain-language explanation of what triggered the warning (the Logic tab opens by itself when the selected node's split needs a review).

The tool applies different logic depending on how you chose to decompose:

Binary splits are proven MECE. If you chose the binary decomposition type and named your two children as opposites, the tool marks both dots green automatically. No heuristic needed — a complement pair is MECE by construction.

Formula splits reconcile. If you set up a formula node (Revenue = Price × Volume), the tool checks that the arithmetic closes. If it does, exhaustiveness is confirmed by the equation. If you add a term that doesn't fit the formula, a warning tells you the equation no longer balances. Formula splits are also checked for **double-counting**: a summed term named like a running "total", or two terms with the same label, is flagged — the numeric version of an ME violation.

Segment splits require an "Other" bucket. If you chose the segments decomposition type and haven't included an "Other" child, the CE dot flags it. The tool isn't insisting on a specific answer — it is making the gap deliberate. One click on *Add an "Other" bucket* (offered on the Logic tab and in the review dock) closes it; the warning then clears itself, because the check is recomputed live from the tree.

Process and framework splits get guidance, not a silent pass. Neither type can be *proven* exhaustive, so instead of an unchecked grey dot the tool asks the right question in words: does the process run end to end, nothing before the first stage or after the last? Does the framework leave anything important outside its categories? Exhaustiveness stays your call — but the question is put to you explicitly.

Looser splits use a shared-word overlap heuristic. For freeform and framework splits, where the structure is more open, MECE Studio looks for children whose labels share significant words — "Digital Marketing" and "Marketing Spend" would trigger an ME warning that **names the colliding pair**. The heuristic deliberately ignores generic or placeholder words (things like "other", "costs", "issues"), and a word that *every* sibling shares is treated as the split's dimension rather than an overlap — so "EU revenue / US revenue / Asia revenue" doesn't flag itself.

Mixed axes are called out. The classic slip from the *Mixed dimensions* pattern above has its own check: siblings that each *name a different way to cut* the level — "By region / By segment / By quarter" — are flagged with a prompt to pick a single axis per level. It is deliberately conservative, firing only when the branches literally state differing axes.

You can name the dimension a split cuts on. In the **Logic** tab, give each split its single axis — *by geography*, *by customer type*, *by stage* — with one click on the common ones. One consistent dimension per level is the backbone of a MECE split, and naming it keeps you honest; the named axis shows on the node, in the synthesis, and in the AI-critique prompt.

The value of the tool here is not that it makes the MECE decision for you — it doesn't. It makes gaps and overlaps **visible** so you decide deliberately. A red dot is an invitation to think, not a veto. You may look at a warning and conclude the split is good enough for your purpose — there is no "dismiss"; the flag simply stays in the tree's review list (below) until the split is genuinely MECE, so a choice to live with it remains visible rather than buried. The discipline is in making that choice consciously rather than missing the problem entirely.

Reviewing the whole tree at once

Per-node dots are perfect while you are building one branch, but on a large tree you don't want to hunt for the red ones. MECE Studio rolls every split's status up to the **tree level**.

A **MECE health chip** sits in the header. It reads **✓ MECE clean** when every split passes, or **⚠ N to review** when some don't — so you know at a glance whether the tree still has open MECE questions, without scanning it node by node.

Click the chip to open the **review dock** — a triage panel over every flagged split. Flags are **grouped by axis** (*Overlaps* — not mutually exclusive; *Gaps* — not collectively exhaustive) and **ranked by branch priority**, so the splits on your 80/20 branches surface first; each row shows the plain-language reason and the branch's priority band. For each one you can:

- **Locate** it — one click centres that node on the canvas and, while the dock is open, dims the clean splits and amber-dashes the edges of the flagged ones, so the problems literally stand out.
- **Review logic →** — jump straight to that node's Logic tab and think the split through.
- **Remedy** a gap — for a missing-bucket (CE) gap, one click adds the right fix: an *"Other"* bucket for a segmentation, or a fresh sub-issue otherwise.

There is no "resolve" or "dismiss" button, by design: the warnings are computed live, so a split leaves the list the moment it is actually MECE — never because you silenced it. The dock is the fastest way to walk a finished tree and close its last gaps before you present it.

Coaching, kept separate from the health chip

MECE Studio draws a deliberate line between two kinds of feedback. The **ME / CE checks** — the dots, the health chip, the review dock — answer exactly one question: is this split mutually exclusive and collectively exhaustive? Nothing else is allowed to move that needle.

Everything else the tool notices about the *quality* of your thinking lives in a separate **Coaching** callout in the inspector, shown as light `info` nudges — never as MECE flags, and never counted in the health chip. These are drawn from the wider problem-structuring canon rather than from MECE itself:

- a branch that is a bare one-word label where a full question, action, or hypothesis was meant;
- a **laundry list** — a split with more than about seven branches, a sign the level wants regrouping into a cleaner cut;
- an **altitude outlier** — one branch pitched at a very different level of abstraction from its siblings;
- a hypothesis still phrased as a question rather than a claim;
- a weak key question, and the why/how consistency check from *Start with the question*.

The separation is the point. If these softer nudges counted toward the health chip, "⚠ 3 to review" would stop meaning "three genuine MECE problems" and start meaning "three things the tool has an opinion about" — and you would soon learn to ignore it. Coaching *advises*; the MECE chip *adjudicates*. Keeping them apart is exactly what lets the chip stay worth trusting.

A worked example

You are investigating "Why are profits falling?" Your first split is Revenue / Costs.

- ME check: revenue and costs do not overlap. Green.
- CE check: Profit = Revenue − Costs; the equation accounts for everything. Green.

You now split Revenue into "European customers / U.S. customers / digital sales."

- ME check: a U.S. customer buying through your digital platform belongs to both "U.S. customers" and "digital sales." Red. The dimensions are mixed — geography and channel.
- Fix: choose one dimension. Either split by geography (Europe / Americas / Asia-Pacific / Other) or by channel (Direct / Digital / Wholesale / Other), then break down the other dimension one level deeper if needed.

You now split Costs into "Labour / Materials / Facilities."

- CE check: what about software licences, logistics, depreciation? If those are meaningful at your scale, the split is not exhaustive. You add "Other" as a placeholder and flag that it needs breaking open if it turns out to be significant.

Walking the tree this way — split by split, ME test then CE test — takes minutes and catches the errors that derail analysis for days.

Summary

MECE is judged split by split, not across the whole tree. Mutually exclusive prevents double-counting; collectively exhaustive prevents missing the answer. Test each split with two quick questions: could something belong to two of these, and is there anything that fits none of them? Aim for MECE, accept a useful near-MECE split over a perfect unusable one, and use an explicit "Other" bucket rather than leaving a silent gap. MECE Studio's live indicators, the Logic tab's explanations, and the tree-level review dock make violations visible as you build — the decision is always yours, but the problem is no longer hidden.

In *Ways to decompose*, you will see how choosing the right type of split — binary, segments, formula, process, or framework — makes MECE easier to achieve from the start.

Ways to decompose

Knowing that a split should be MECE is one thing. Knowing *how* to cut it is another. Most practitioners who struggle with issue trees are not confused about the MECE principle — they are stuck staring at a blank node, wondering which children to draw. This chapter gives you a small toolkit of reliable decomposition types, a way to choose among them, and a feel for what each one looks like in practice.

The six types are: **binary**, **segments**, **process**, **formula**, **framework**, and **freeform**. None is universally right. Each is well-suited to a particular kind of question, and each has a different relationship to MECE.

Binary (A / not-A)

A binary split divides the world into something and its complement: churned customers vs. customers who did not churn; products above target margin vs. products below; revenue from new channels vs. revenue from existing channels.

Binary is always MECE. The logic is airtight: something either belongs to the A group or it doesn't, there is no third option, and together A and not-A cover everything. You cannot overlap, and you cannot leave a gap.

The catch is that the two labels have to *be* a complement, not just two things. "Above target margin / below target margin" looks binary, but a product exactly on target falls in neither. MECE Studio only marks a binary split as provably MECE when the wording shows the complement: one branch negates the other ("churned / did not churn"), or they form a known pair ("new / existing"). Anything else it reports as not proven, which is your cue to reword.

Use binary when:

- You are not sure yet how to cut a question and need a safe start.
- You want to isolate a group you care about while keeping everything else in view. "Is the profit problem in the revenue line, or is it not in the revenue line?" forces you to look both places.
- You want to make sure you are not ignoring the other half. Binary makes the complement explicit, which is its greatest virtue. Teams routinely investigate the A side and forget the not-A side exists.

The limitation is precision. "Customers who did not churn" is a real and useful category, but it is also enormous and heterogeneous. Binary is typically a first cut that you then refine on one or both sides.

Running example. You suspect the profit decline is a revenue problem. A binary split of the root question: **Revenue is falling / Revenue is not falling**. If Revenue is fine, the whole problem is on the cost side and you can stop investigating revenue. If revenue is falling, you go one level deeper.

Segments

A segment split divides a whole into its parts: geographic regions, customer types, product lines, business units, time periods. You are carving up a population or a total.

Segments are MECE only with an explicit **"Other"** bucket. This is the standard trap: you list the segments you know (Europe, Americas, Asia-Pacific) and implicitly assume nothing else exists. If even one customer

or one dollar of revenue is in a country that fits none of those three, the split is not exhaustive. Adding "Rest of World" or "Other" makes it honest.

Use segments when:

- The question is naturally about parts of a whole: "where is the revenue concentrated?", "which customer group is driving churn?", "which product line is underperforming?"
- You have data already segmented this way and want the tree to match reality.
- The problem is likely to be concentrated in one part, and you want to isolate it.

Watch for the mixed-dimension trap described in *MECE: no overlaps, no gaps*. "Big / small / online" mixes size and channel. Pick one dimension at a time and split cleanly on that dimension before adding another.

Running example. Revenue splits into: **Domestic / International / Other**. Each region is a distinct segment; Other captures anything outside the main two. Now you can investigate whether the drop is concentrated in one region, or spread evenly, before going deeper.

Process / stages

A process split follows the order of a sequence: the stages of a sales funnel, the steps in a supply chain, the phases of a customer journey. You are asking "at which stage does the problem occur?"

Process splits are MECE when the stages are ordered (each follows the last), non-overlapping (a unit is at one stage at a time), and complete (the sequence runs from start to finish with no steps missing). A classic funnel — Awareness → Consideration → Trial → Purchase → Retention — is MECE if those five stages cover the entire customer relationship.

Use process when:

- The question is explicitly about a pipeline, a journey, or a workflow: "where is the sales process leaking?", "at which stage are customers dropping off?", "which step in the production process creates the defect?"
- You need to assign ownership, because stages often map to teams.
- The goal is to isolate the bottleneck — which is much faster when you can rule out entire stages.

The risk is gaps in the sequence. If your customer journey has a post-purchase onboarding phase that matters but you omit it, you have a CE failure. Map the full process first, then decide which stages to group or split.

Running example. You are investigating why revenue from new customers is falling. You split the sales process into: **Lead generation / Qualification / Proposal / Close / Onboarding**. Each stage is ordered, non-overlapping, and the five stages cover the full journey from stranger to active customer. You can now ask at which stage the conversion rate has changed.

Formula / value-driver

A formula split decomposes an outcome using arithmetic: Revenue = Price × Volume; Profit = Revenue − Costs; Customer lifetime value = Average order value × Purchase frequency × Gross margin.

Formula splits are provably exhaustive because the equation is the proof. If the equation holds, you cannot have a gap — every contributor to the outcome appears as a term. And the terms are exclusive by definition,

because each represents a distinct quantity.

Use formula when:

- The question is quantitative: "why is revenue below target?", "what is driving margin compression?", "how sensitive is the model to price changes?"
- You want to anchor the tree in arithmetic that can be measured and stress-tested.
- You are building toward a value-driver tree where numbers on the leaves roll up to the root.

The mechanics of value-driver trees — how to assign values, units, and operators, how numbers roll up, and how to run a sensitivity analysis — are covered in *Doing the numbers*. What matters here is the structural point: writing the equation first gives you the split for free, and the split is MECE by construction.

Running example. Revenue splits into **Price** and **Volume** (Revenue = Price × Volume). You can immediately ask: has average selling price fallen, or have units sold fallen, or both? Those are different investigations, requiring different data and pointing to different remedies.

Framework

A framework split uses an established lens: People / Process / Technology; the 4 Ps (Product, Price, Place, Promotion); McKinsey's 7-S; Porter's Five Forces; the 3 Cs (Company, Customers, Competitors). Someone else has already done the decomposition work, and you are borrowing it.

Frameworks are fast and give you something credible to show stakeholders who recognise the lens. The risk is that a generic framework may not be MECE for your specific problem. The 4 Ps were designed for marketing; applied to "why is employee retention falling?", Price and Place are at best a stretch. Before committing to a framework split, run the standard ME and CE tests: would anything fall between the categories? Can anything land in two?

Use frameworks when:

- The problem is well-trodden and the framework was designed for it.
- Speed matters more than precision — a framework gets you to a draft tree in minutes.
- You need to communicate the structure to a broad audience and the framework carries meaning for them.

Always check the fit explicitly. A framework is a starting point, not a certification of MECE.

Running example. You split the cost question using People / Process / Technology. Labour costs sit in People; inefficient workflows in Process; software and equipment in Technology. Check: is there anything that fits two buckets? Consulting fees paid to fix a technology problem could land in either. Either rename the buckets or accept the overlap and note it. Check: is there anything that fits none? Rent, raw materials, energy — none of the three standard categories clearly owns these. Add a fourth, or switch to a formula split of the cost line instead.

Freeform

Freeform is what you do when none of the structured types fits yet. You name children as they occur to you, without committing to a structural principle. It is brainstorming, not decomposition.

Freeform is a legitimate early step. Messy problems rarely announce their structure upfront. Writing down four or five possible drivers as freeform children is often the fastest way to see whether they cluster into a cleaner type.

Use freeform when:

- You genuinely do not know how the problem divides yet.
- You are exploring with a client or team and want to capture ideas before organising them.
- You are mid-tree and have hit a node that resists a clean structural cut.

The discipline is to treat freeform as temporary. Once you have the children on the canvas, ask: do these cluster? Do they follow a sequence? Can I write an equation? Could I split binary first? If the answer to all of those is no, freeform may be the right permanent answer — but you should have asked.

How to choose

Start from how the thing naturally divides.

If the question is quantitative, write the equation. Formula.

If you can identify a sequence, map the steps. Process.

If you are looking at parts of a population or total, decide on one dimension and segment. Add Other.

If you are stuck, go binary. It is always MECE, it forces you to consider the complement, and it gets something on the canvas immediately.

If the domain is well-understood, borrow a framework. Check the fit.

If nothing else works, start freeform and look for structure after you have the ideas out.

A single tree can mix types across levels. The root might split by formula (Profit = Revenue − Costs), Revenue by segment (Domestic / International / Other), and the domestic revenue shortfall by process (at which funnel stage is conversion falling?). The rule is that each individual split uses a coherent type, not that the whole tree must use one.

Scaffolds in MECE Studio

When you add a child node in MECE Studio and choose a decomposition type, the tool seeds **scaffold** starter children appropriate for that type:

- **Binary** → two children labelled with a placeholder and its complement, ready for you to rename.
- **Segments** → two placeholder segments plus an "Other" bucket, already satisfying the exhaustiveness requirement for segment splits.
- **Process** → a short sequence of placeholder stages.
- **Formula** → placeholder terms reflecting a simple equation structure.

A scaffold is a prompt, not an answer. The labels are generic; your job is to rename them to reflect the actual question. The point of the scaffold is to remove the blank-canvas problem — you always have something to react to — and to make the structural requirement of the type concrete. A segments scaffold that includes

"Other" from the start reminds you that exhaustiveness requires it, even before you have filled in the real segments.

You can ignore the scaffold and type your own children directly. The decomposition type still governs the MECE check, so a segments node without an Other child will still flag the CE warning. The scaffold just makes it easier to start right.

The same idea applies from the very first move: when you build a tree from the Start page's key-question box, MECE Studio asks **"how do you want to split it?"** — pick a type and you land on a scaffolded, already-checkable first split instead of a lone root box (or start blank and decide later). Choosing the cut is the first analytical decision, and the tool puts it where it belongs: up front.

Named frameworks in MECE Studio

Beyond the generic scaffolds, the **Templates** page carries a small library of **named frameworks** — the established lenses from the Framework section above, ready to drop onto a blank tree with their canonical branches already filled in. Pick one and you get a starter tree to rename to your situation:

- **Marketing** — the 4 Ps (Product / Price / Place / Promotion) and Lauterborn's customer-centric 4 Cs (Consumer wants / Cost / Convenience / Communication).
- **Strategy and industry** — Ohmae's 3 Cs (Company / Customers / Competitors), Porter's Five Forces, PESTEL, SWOT, and the BCG and Ansoff matrices.
- **Organisation** — McKinsey's 7-S.
- **Growth and diagnosis** — the AARRR "pirate metrics" funnel and the Ishikawa fishbone (the 6 Ms) for root-cause work.

Two things are deliberate about how these behave. First, they are *starters*, not answers — the root is a placeholder you rename, and every branch is yours to adapt. Second, and more important, MECE Studio does **not** badge them as provably MECE. They are typed as framework (or process, for the AARRR funnel), so the tool reports their exclusivity and exhaustiveness as *unchecked* rather than guaranteed. This is honest by design. None of these famous lenses is a clean partition: PESTEL's Political and Legal factors overlap, SWOT is a discussion starter that makes no claim to be exhaustive, and Porter built the Five Forces precisely because he found SWOT lacking in rigour. The library gives you the speed of a recognised framework while the checks keep reminding you to verify the fit — which is exactly the discipline this chapter argues for.

Summary

Six decomposition types, each with a different MECE relationship:

Type	MECE?	Best for
Binary	Always	Stuck; want to isolate A and see not-A
Segments	With "Other"	Parts of a population or total
Process	When complete	Pipeline, funnel, journey
Formula	By construction	Quantitative outcome; value-driver tree

Type	MECE?	Best for
Framework	Check the fit	Well-trodden domain; speed
Freeform	Rarely	Early exploration; treat as temporary

Choose based on how the thing naturally divides. Prefer binary when stuck. Use formula whenever the question is quantitative, and read *Doing the numbers* to take it further. Use scaffolds as a starting point in MECE Studio — rename freely, but notice what the structure is telling you.

With the structure standing, the next move is to commit to a working answer and let the tree test it. That is the job of *Working hypotheses* — and *Prioritise* then decides which branches to work first.

Working hypotheses

Most people approach a complex problem the same way: map everything out, gather data on every branch, and wait for an answer to crystallise. It feels thorough. It's also one of the slowest paths to insight — and one of the most common reasons analyses run out of time or lose their audience.

Hypothesis-driven problem solving flips the sequence. Instead of analysing first and concluding later, you commit to a **candidate answer** early and use the tree to test it. The tree stops being a map of everything that might matter and becomes a targeted argument you are trying to break.

This chapter explains how to work that way and how to avoid the traps that come with it.

What a hypothesis is

A **hypothesis** is a specific, falsifiable claim about your key question. Not a vague direction — a claim precise enough that evidence could prove it wrong.

"Profits are falling because our cost base grew faster than revenue" is a hypothesis. "There might be a cost issue" is not.

The best hypotheses have two properties:

1. They are **concrete** enough to point at specific data. You know what you would need to see to confirm or refute them.
2. They are **bold** enough to commit to a direction. A hypothesis that covers every possibility is just a restatement of the question.

> *A good hypothesis is a bet. The purpose is not to be right on day one — it is to be wrong efficiently.*

Why start with a hypothesis at all?

The logic is counterintuitive but holds up in practice. When you have a hypothesis, every branch of the tree is doing one of two things: providing evidence that the hypothesis is correct, or providing evidence that it is wrong. Both outcomes are useful. Analysis without a hypothesis rarely achieves either — it accumulates facts without accumulating conviction.

There are three concrete gains.

Focus. A hypothesis tells you which branches carry the answer and which are background noise. In the profits-falling tree, if your hypothesis is "revenue is flat but volume mix shifted toward lower-margin products," you immediately know which sub-issues to develop and which to treat as secondary. You don't spend three weeks building an exhaustive cost model if costs aren't the story.

Speed. Working consultants and analysts often have days, not months. The hypothesis gives you the fastest path to either a defensible conclusion or a clear statement of why the original guess was wrong. Both end the right way.

Communication. An audience — whether your manager, a board, or a client — finds it far easier to engage with "here is our working answer, here is why we believe it, here is what would change our mind" than with

"here is everything we found." The hypothesis frames the conversation before you open a single slide.

Building the day-one hypothesis

A **day-one hypothesis** is your best early guess at the answer, formed before you have done deep analysis. It draws on whatever you already know: prior experience with similar problems, a quick scan of available data, intuition from people close to the situation.

This is legitimate and important. You are not supposed to arrive at the problem blank. The day-one hypothesis is the raw material the tree will sharpen or discard.

For the profits question, a day-one hypothesis might be: "Gross margin per unit is being squeezed by rising input costs, while pricing has stayed flat." That's specific enough to test. It points at input-cost data and pricing history. It ignores headcount and fixed overhead for now — not because those don't matter, but because your initial read says they're not the driver.

You write this down and make it explicit. One sentence on the whiteboard. In MECE Studio, that sentence has a dedicated home: the **Answer** banner above the canvas holds the governing answer the whole tree argues for. State the day-one hypothesis there, and the synthesis (see *Answer-first*) will later open with it and a verdict on how well it held up. Then you build the tree to test it.

Turning sub-issues into testable claims

The tree in *The issue tree* chapter showed how to decompose a key question into sub-issues. In a hypothesis-driven tree, each sub-issue becomes a **testable claim**, not just a category.

Instead of: "What happened to gross margin?" → sub-issues: volume, price, cost.

You write: "Gross margin declined because input costs rose ~15% over the period while selling prices held flat."

Now the sub-issue is a claim. You know what confirming it looks like (input cost data showing a 15% rise, pricing data showing stability). You know what refuting it looks like (costs flat, or prices declining in step with costs). The branch has direction.

This is the practical mechanic of hypothesis-driven work: at each level of the tree, the split isn't just "what are the components?" but "what do I believe about each component, and what would change my mind?"

Using MECE Studio to track status

As you gather evidence and reasoning, each node in your tree is in one of four states:

- **Open** — the claim is still live; you haven't confirmed or ruled it out yet.
- **Supported** — the weight of evidence points toward this being true.
- **Refuted** — the evidence is against it; this branch is no longer carrying the answer.
- **Parked** — possibly relevant, but you've decided not to pursue it in this analysis.

MECE Studio lets you set this status on each node. The node displays a colour-coded edge so you can scan the whole tree and immediately see which branches are alive, which have been disproved, and which you're

not pursuing. When you have fifteen nodes in the tree, this visual at-a-glance state is what keeps the team oriented.

The key mental shift: **a refuted branch is progress, not failure.** If your day-one hypothesis was "input costs are the driver" and you now have solid data showing costs were flat, you have learned something real. You mark that branch refuted, and the tree tells you where to look next. Elimination is half the work of structured problem solving.

Parked is also a legitimate status. Sometimes a branch is genuinely relevant to a deeper question but isn't the right fight for this engagement or timeline. Marking it parked is honest — you're not pretending it doesn't exist, you're making a conscious resource decision. MECE Studio keeps it visible so it doesn't get forgotten.

The discipline: try to kill your hypothesis

This is where most people go wrong. Once you've formed a hypothesis and started building support for it, there is a strong pull toward confirming rather than testing. You notice the data that fits, you underweight the data that doesn't, you frame questions in ways that lead witnesses toward the answer you expect.

This is confirmation bias in its most professionally damaging form. It produces analyses that feel airtight right up until someone in the room asks the one question you didn't pursue — and the whole argument collapses.

The antidote is a standing discipline: **actively look for the thing that would kill your hypothesis.**

Ask yourself explicitly: what data, if I saw it, would force me to abandon this branch? Then go look for that data. If you can't find it, that is genuine support. If you do find it, you've saved everyone a lot of time.

Practically, this means:

- Identifying the **critical assumption** your hypothesis rests on — the one thing that has to be true — and testing it first.
- Seeking out people who disagree and understanding their reasoning.
- Treating contradicting evidence as higher priority than confirming evidence, because it's rarer and more valuable.

If your hypothesis is "revenue is falling because of customer churn," the critical assumption is that the customer base is actually shrinking. Before you build a fifty-slide churn analysis, check that assumption. If retention is actually flat and the revenue decline is from lower order values among existing customers, you've just redirected the whole analysis in an hour.

Don't fall in love with the first hypothesis

The day-one hypothesis is a starting position, not a conclusion. The tree may evolve it significantly.

Common patterns:

- The hypothesis is **directionally correct but misspecified**. Costs are the driver, but it's logistics costs, not input costs. You refine rather than discard.
- The hypothesis is **correct but incomplete**. Churn is real, but so is price erosion. The answer requires two branches.

- The hypothesis is **simply wrong**. The data contradicts it cleanly. You shift to an alternative, using what you now know.

All three of these are fine outcomes. What you want to avoid is carrying a refuted hypothesis forward because you've invested in it. The colour-coded tree helps here — it makes it hard to pretend a red branch is green. When three of your five sub-issues are marked refuted, the hypothesis supporting them needs to change.

In MECE Studio you can also **park** a hypothesis you've moved away from without deleting it. That matters if someone asks later why you didn't pursue a particular direction — you have a record that you considered it and when you set it aside.

A note on sequencing

The chapters *Prioritise the 80/20* and *Evidence* build directly on what's here. Prioritisation helps you decide which branches to test first — the answer is almost always: test the critical assumption of your current hypothesis before you test anything else. Evidence is the mechanism by which nodes move from open to supported or refuted.

Together, these three practices — hypothesis formation, prioritisation, and evidence gathering — are the operating cycle of a hypothesis-driven analysis. You form a belief, you decide what to test, you gather what you need, and the tree reflects reality more accurately with each iteration.

The tree at the end of a good analysis looks very different from the tree at the start. Many branches are refuted. A few are supported. The synthesis almost writes itself, because the answer is visible in the structure before you put words to it. That is the goal — not a perfect first hypothesis, but a disciplined process that converts a messy question into a clear conclusion, fast.

Prioritise the 80/20

A well-built issue tree can have fifteen, twenty, thirty nodes. You cannot work all of them equally. And if you try, you will spend three weeks doing analysis that a sharper team would finish in four days — and you'll probably reach a weaker conclusion.

Prioritisation is how you decide where to spend the hours you actually have.

The core lens: impact × ease

Every branch of the tree can be rated on two dimensions.

Impact is how much working this branch could move the answer. If this sub-issue turns out to be the driver, how significantly does it explain what you're investigating? For the profits question: if gross-margin compression on the top-five products is responsible for 80% of the profit decline, that branch has enormous impact. The branch tracking administrative overhead on a segment that represents 3% of revenue has very little.

Ease is how cheap the branch is to test. Cheap means: the data is already available, the analysis is quick, and the conclusion is clear. Expensive means: you'd need weeks of fieldwork, primary research, or modelling to reach a conclusion.

The rule is blunt: **pursue high-impact, high-ease branches first.** They give you the most answer per unit of effort. If you're lucky, one or two of these will crack the problem wide open before you've touched the difficult branches at all.

> *The point of prioritisation is not to do less work. It is to do the right work before you run out of time.*

The 80/20 instinct

The Pareto principle is a pattern, not a law — but it holds surprisingly often in structured problem solving. A small number of branches usually carry most of the explanatory weight.

In practice: of a twenty-node tree, three or four nodes typically account for the bulk of the answer. The rest is context, nuance, or simply not the story this time. If you let yourself get pulled into equal treatment of all branches, you dilute your resources across work that doesn't move the conclusion.

The 80/20 instinct means looking at your tree at the start of each day and asking: if I could only work three branches today, which three would get me closest to an answer? Then working those three, not the five easiest, not the ones you happen to find interesting, and not the ones your stakeholder mentioned last.

How to score branches

You don't need a complex model. A simple 1–3 rating on impact and a 1–3 rating on ease gives you nine cells. The top-right corner (high impact, high ease) is your immediate priority. The bottom-left corner (low impact, hard to test) can often be dropped entirely.

In MECE Studio, you set a branch's priority in one click on a **3×3 impact-by-ease matrix** in the inspector — the nine cells of exactly this scoring model — with the resulting **High / Medium / Low priority band** shown live as you pick, and as a chip on the node itself, so when you look at the canvas you can see at a glance which branches are tier-one work and which are background. (On the canvas, pressing P on a selected node cycles its priority without opening the inspector.) When you export or generate the synthesis, MECE Studio orders the findings by priority — the high-impact nodes come first in the narrative, which is usually also the right structure for the answer.

Scoring doesn't have to be precise. A rough calibration across the team ("we all agree gross-margin compression is high impact; we all agree the admin overhead branch is low impact") is good enough to direct effort. The value is the shared prioritisation decision, not the score itself.

The order branches sit in

Priority decides which branches you *work* first. By default it also decides the order they are *read* in — on the canvas and in the synthesis, heaviest branch leading. But importance is not the only order that makes sense for a set of siblings. Barbara Minto identifies three ways to order a group of ideas, and a good tree uses whichever one fits the split:

- **By importance** — most significant first. The default, and the right choice for a diagnostic split where you want the 80/20 branch to lead.
- **By time** — the order in which things happen. A process split — Lead → Qualify → Propose → Close — reads as nonsense sorted by priority; its meaning *is* the sequence.
- **By structure** — a fixed partition that mirrors something in the world: geography (North / South / East / West), an org chart, a product hierarchy. Re-sorting it by priority just makes a branch harder to find.

Getting this right is a small thing that makes a tree much easier to read: the eye expects a sequence to run in sequence and a ranking to run by rank.

MECE Studio lets you set it per split. The global **Sort siblings by priority** setting orders every split by importance; but on the inspector's **Logic** tab a split can declare its own **ordering principle** — *importance*, *time*, or *structure* — which overrides the global setting for that split alone. Mark a process split *time* and its stages stay in sequence even when the rest of the tree is sorted by priority; mark a fixed partition *structure* and it keeps the order you authored. It is the one place a split gets to say "don't re-sort me" — so a funnel or a regional breakdown reads the way it should, while your diagnostic splits still lead with their heaviest branch.

The branches you should kill early

Some branches in a tree are there because MECE discipline required them — they complete the exhaustive coverage of the space — not because you expect them to carry the answer.

Killing these branches early (marking them low priority and moving on) is a feature, not a shortcut. You are not ignoring them; you are making an explicit, informed choice to treat them as background. MECE Studio's parked status in the hypothesis workflow (see *Working hypotheses*) handles this case — a branch can be structurally present without consuming analysis hours.

The test: if this branch turned out to be the driver, how surprised would you be? If the answer is "extremely surprised," it's a candidate for early deferral. If the answer is "not very surprised," it probably deserves time.

The fairness trap

The most common prioritisation failure is spending equal time on every branch because it feels rigorous or fair. It is neither.

A team that allocates one week to each of eight branches will produce eight shallow analyses and a thin conclusion. A team that allocates six weeks to the three branches most likely to carry the answer and two weeks across the rest will produce a real finding.

Equal treatment signals an absence of judgment. Stakeholders often read it that way. "We looked at everything" is a weak conclusion. "We identified the key driver, confirmed it with data, and ruled out three alternative explanations" is a strong one.

The other version of the fairness trap: spending disproportionate time on the branch that's easiest to staff or most interesting to the team, regardless of its likely impact. The availability of data should influence your ease score, not your impact score. Don't let easy-to-access work crowd out the harder inquiry on the branches that actually matter.

Sequencing your hypothesis tests

Prioritisation and hypothesis-driven analysis (see *Working hypotheses*) work together most powerfully when you sequence your hypothesis tests by impact.

Your day-one hypothesis rests on at least one critical assumption. Find that assumption, check its impact score, and test it first. If it's high impact and the data is available, you should be able to confirm or refute the core of your hypothesis within the first day or two of analysis.

This is not always possible — sometimes the critical assumption requires fieldwork or modelling that takes time. In that case, run the high-ease / high-impact branches you can test quickly in parallel, so the team is generating findings while the harder test is running.

The goal: by the midpoint of any analysis, you should have a defensible position on the question. Not a final answer, but a working answer that would hold up to scrutiny. Prioritisation is what gets you there.

When to revisit priorities

Priorities are not fixed. As branches get resolved — either supported or refuted — the landscape changes. A refuted high-priority branch shifts priority to whatever the next-best hypothesis requires. A surprising finding on a low-priority branch might suddenly make adjacent low-priority branches more interesting.

Revisit the priority scores briefly at natural checkpoints: when a major branch resolves, when you get a large new data set, when a stakeholder conversation changes your read on the problem. Don't re-score the whole tree every morning — that's analysis paralysis applied to the meta-process. But don't let a priority allocation made on day one go unexamined for three weeks either.

The summary rule

If you take one thing from this chapter: bias heavily toward the branches most likely to crack the problem, test them early, and be willing to abandon low-value work rather than carrying it out of completeness anxiety.

A complete tree with twenty-five evenly-worked branches and a vague conclusion is not rigorous. A tree where twelve branches are parked or refuted and three are deeply evidenced, pointing clearly to a finding, is what rigour actually looks like.

The 80/20 principle won't be precisely 80/20 in your specific problem. But the underlying truth holds: a few branches carry most of the answer. Find them fast, work them hard, and let the rest wait.

Priorities decide *where* the hours go. The next chapter — *Evidence* — is about what those hours actually produce: the facts that move a branch from a bet to a finding.

Evidence

A hypothesis is a claim. Evidence is what transforms a claim into a conclusion. Without it, your issue tree is an organised speculation — useful for structuring the problem, not useful for answering it.

This chapter is about how evidence works in structured problem solving: where it comes from, how to weigh it, and why the evidence that challenges your hypothesis is worth more than the evidence that supports it.

Two directions of evidence

Every piece of evidence does one of two things relative to a hypothesis: it either **supports** it or **contradicts** it.

Supporting evidence is consistent with the claim being true. Contradicting evidence is consistent with the claim being false. Both are useful; both belong in the analysis.

This sounds obvious. In practice, most analysts weight supporting evidence heavily and let contradicting evidence slip away. They remember the interview that confirmed their view and forget the one that didn't. They pull the data that fits and treat the data that doesn't as an anomaly worth explaining later.

That is confirmation bias, and it produces conclusions that collapse under scrutiny. The discipline of working with evidence starts with committing to log both directions honestly.

> *Contradicting evidence is not a problem to be explained away. It is information the tree needs.*

Not all evidence is equal

A second dimension matters as much as direction: **strength**.

Consider two pieces of supporting evidence for the claim "gross margin compression is being driven by rising input costs":

1. A cost controller mentions in passing that "things feel more expensive lately."
2. A line-by-line comparison of input invoices from Q1 last year versus Q1 this year shows a 14% weighted average cost increase across the top-ten materials.

Both support the hypothesis. They do not carry equal weight. The second piece should move your confidence substantially. The first should move it only a little.

The practical categories are roughly:

- **Weak**: anecdote, single observation, unverified secondhand account, intuition of a knowledgeable person.
- **Moderate**: directional data from a limited sample, a consistent pattern across several interviews, internal benchmarks.
- **Strong**: clean quantitative data covering the relevant population, multiple independent sources converging, external benchmarks from comparable organisations, primary research with statistical validity.

You don't need a precise formula. What you need is an honest read: is this the kind of evidence that would hold up if someone challenged it in front of a sceptical audience?

In MECE Studio, when you attach evidence to a node you choose its **strength up front** — anecdote, indicative, or strong — along with its direction (supporting or contradicting). As the picture sharpens you can switch an item's strength with a direct picker, **flip it between supporting and contradicting** in place, or click its text to edit it. The node displays a count badge showing how many supporting and contradicting items you've gathered. This makes the state of evidence visible at a glance — a node with five strong supporting items and zero contradicting items is in a different position from a node with two weak supporting items and one moderate contradicting item.

The special value of disconfirming evidence

One solid contradicting fact is often worth more to your analysis than ten confirming ones.

This is not intuitive, but the reasoning is sound. If your hypothesis is correct, you expect confirming evidence to show up almost everywhere you look. It's the default. The absence of contradicting evidence in a thorough search is meaningful; the presence of confirming evidence in an unstructured search is not.

Contradicting evidence, by contrast, is harder to find because you tend not to look for it — and when you do find it, it's diagnostic. One data point that clearly contradicts your hypothesis can:

- Prompt you to refine the hypothesis (the claim was too broad)
- Reveal a boundary condition you hadn't accounted for
- Kill the branch entirely, saving you from building a case that will fall apart later

The practical implication: when you are gathering evidence for a branch, deliberately search for the contradicting case. Ask the interviewee who is most likely to see it differently. Pull the numbers for the time period or segment most likely to look wrong. If you find nothing, that is genuine support. If you find something, you needed to find it.

Practical evidence sources

Evidence doesn't require a research budget. It requires looking in the right places.

Internal data is usually the first stop: sales figures, cost ledgers, operational metrics, historical comparisons. These are often imperfect — incomplete, inconsistently defined, covering the wrong time horizon — but they are fast and free. The question is not whether the data is perfect but whether it is directionally reliable enough to move your belief about the hypothesis.

Interviews are underused in quantitative-heavy teams and overused in qualitative-heavy ones. The right framing: interviews are evidence sources, not proof. A consistent pattern across six independent interviews — where each person is describing the same dynamic from their own vantage point — is moderate-to-strong evidence. A single interview where someone confirms what you already believe is weak evidence, regardless of how senior the person is.

Observation means looking at what actually happens rather than what people say happens. For operational questions especially — "why is the process slow?", "where are errors introduced?" — watching the process

is often faster and more accurate than interviewing people about it. Walk the floor. Run a transaction yourself. The gap between described process and actual process is frequently where the answer lives.

External benchmarks answer the question "is this unusual?" Your cost base grew 12% — is that fast or slow relative to comparable organisations? Without a benchmark, you can't calibrate. Industry reports, public financial filings, third-party databases, and published research all provide reference points. Benchmarks are often the fastest way to move a branch from "possibly the driver" to "definitely the driver" or "probably not the driver."

How evidence moves a node from open to resolved

The mechanism is cumulative and a matter of judgment, not formula.

A node starts **open**: you have a claim and no evidence. As you gather evidence, you are building a case. At some threshold — when the weight of evidence is clear enough that a reasonable sceptic would accept the direction — the node moves to **supported** or **refuted**.

In MECE Studio, you make this call explicitly. You look at the evidence attached to the node: how many supporting items, how many contradicting, at what strength levels. You consider whether the contradicting evidence has been investigated and explained or whether it remains a genuine challenge. Then you set the status. The colour-coded edge reflects your judgment.

This is deliberate. There is no algorithm that sets the status automatically, because the judgment requires understanding the evidence, not just counting it. Two weak supporting items and one moderate contradicting item might resolve differently depending on whether the contradicting item has a known explanation or not.

What MECE Studio does is ensure you do not skip this judgment. Every node in the tree has a visible status, and an open node with accumulated evidence is a flag: you have what you need to call it, but you haven't called it yet.

The confirmation bias trap in practice

Confirmation bias shows up in evidence work in specific, recognisable patterns:

- You interview the people most likely to agree with you.
- You pull data for the period or segment that looks best for your hypothesis and treat other periods as "exceptional."
- You weight an anecdote from a credible senior person heavily because it confirms your view, without noticing you would treat the same anecdote as weak if it contradicted you.
- You mark a branch supported after finding two moderate confirming items, without checking whether there's a strong contradicting data set you haven't looked at.

The most useful counter-habit: before marking any branch supported, ask yourself what the strongest possible case against this branch is, and whether you've actively looked for evidence of it. If you haven't, do that first. The time you spend looking for the thing that kills the hypothesis is almost never wasted — either you find something important, or you close out the branch with much higher confidence.

Evidence and synthesis

Once nodes start moving from open to supported or refuted, the synthesis (see *Answer-first synthesis*) begins to write itself. The answer to the key question is the pattern of supported and refuted branches, read up from the bottom of the tree.

Evidence is what makes that synthesis credible rather than merely plausible. When you present a conclusion and someone asks "how do you know?", the answer should be: this branch is supported by these three items of evidence at these strength levels; this alternative was refuted by this contradicting data; the finding holds up across these independent sources.

That is the difference between a structured hypothesis and a structured answer. The tree gives you the structure. Evidence gives you the substance.

One kind of evidence deserves a chapter of its own: when the question is quantitative, the numbers themselves carry the argument. That is where the next chapter, *Doing the numbers*, picks up.

In MECE Studio, when you run the answer-first synthesis, the evidence attached to supported and refuted nodes is available to pull into the narrative — the ✓ and ✗ counts are a reminder of what's behind each claim. The goal is not to produce an evidence dump, but to make sure every claim in the synthesis has something behind it, and that you know what that something is before you walk into the room.

Doing the numbers

Most problems that land on your desk have a number somewhere at the root. "Why are profits falling?" is not just a structural question — it is a quantitative one. The answer, eventually, has to be: *by how much, driven by what, fixable for how much?*

When your question is quantitative, your issue tree becomes something more specific: a **value-driver tree**. The logic is identical — MECE splits, working hypotheses, prioritisation — but now every split carries an operator, every leaf carries a number, and the whole thing rolls up into the number you care about at the root.

This chapter shows you how to build that kind of tree, read a reconciliation check, and use sensitivity analysis to find the number that actually matters.

Why structure beats a spreadsheet

A spreadsheet can do all of this arithmetic. So why bother with a tree?

Because a spreadsheet hides its logic. You see cells and formulas, but you cannot see at a glance whether the model is MECE, which branch drives the most variance, or where the weak assumptions sit.

A value-driver tree makes the **structure visible**. Every split is justified — you cannot add a row out of habit; you have to explain why it belongs as a child of its parent. The MECE check is live, so overlaps and gaps surface immediately. And the sensitivity ranking is computed from the tree topology, not from a column you manually highlighted.

The practical effect: when you present the numbers, you can defend the structure as well as the arithmetic. That is a different conversation than "trust the model."

Operators: sum, product, difference

Every node in a value-driver tree either carries a leaf value or is defined by its children via an **operator**. There are three that cover almost everything:

Sum — the parent is the total of its children.

> *Gross revenue = Product A revenue + Product B revenue + Product C revenue*

Product — the parent is the product of its children.

> *Revenue = Price × Volume*

Difference — the parent is one child minus another.

> *Profit = Revenue − Cost*

That third one is the split you will use most. Almost every profitability question decomposes, at the first level, into a difference: the top line minus the bottom line.

When you assign an operator to a node in MECE Studio, the tool knows that the node's value is *derived* — you do not enter it by hand. Instead you enter values on the leaves, and the parent is computed. This is what makes roll-up possible.

Building the running example

Take the question from earlier chapters: **"Why are profits falling?"**

The first split is a difference:

```
Profit = Revenue − Costs
```

Revenue decomposes by product:

```
Revenue = Price × Volume
```

So Price and Volume are leaves. Costs decomposes as a sum:

```
Costs = Input costs + Operating costs
```

Both of those are also leaves for now.

Here is the tree with illustrative numbers filled in:

Node	Operator	Value	Unit
Profit	difference	*computed*	DKK
Revenue	product	*computed*	DKK
Price	leaf	250	DKK/unit
Volume	leaf	4 000	units
Costs	sum	*computed*	DKK
Input costs	leaf	620 000	DKK
Operating costs	leaf	380 000	DKK

Roll up: Revenue = 250 × 4 000 = **1 000 000 DKK**. Costs = 620 000 + 380 000 = **1 000 000 DKK**. Profit = 1 000 000 − 1 000 000 = **0 DKK**.

That is a break-even position. Now suppose last year's numbers were: Price 270, Volume 4 200, Input costs 590 000, Operating costs 350 000. Last year's profit: (270 × 4 200) − (590 000 + 350 000) = 1 134 000 − 940 000 = **194 000 DKK**.

The profit has gone from 194 000 to zero. That is the question. The tree tells you the arithmetic is exact — but it does not yet tell you which driver to chase.

Entering values and units in MECE Studio

On a leaf node, open the inspector's **Value** tab and enter a numeric value and a unit. Units matter: they prevent category errors (adding DKK to units), and they appear on exports so your audience can see what they are reading.

On an interior node, the operator (sum / product / difference) lives on the **Logic** tab — alongside the **Roll up children** button and the sensitivity bars — so leave the Value tab's field blank and the tool computes the node from its children. If you do enter a value manually on an interior node, the **reconciliation check** activates.

The reconciliation check

The reconciliation check answers a simple question: *do the children actually add up to what you claimed the parent is?*

This matters because you will sometimes inherit a parent value from an external source — a reported profit figure, a budget line, a benchmark — and build the tree from the top down. You fill in the leaf estimates, roll up, and then check: does the computed value match the external figure?

A tolerance exists because real data is messy. If the mismatch is small (within tolerance), the check passes. If it is large, the check flags it: the children do not reconcile with the parent.

Units are part of the check, not decoration. If terms that get **added or subtracted** carry different units — 60 M DKK plus 40 k DKK — their total means nothing, so the check says so instead of quoting a number. A **product** is exempt: multiplying dimensions is supposed to change the unit, which is exactly what Price (k DKK) × Volume (k units) = Revenue (M DKK) does. Leaving a unit blank means "same as the others", so you only have to type it once.

> *A failing reconciliation is not always a mistake. It is sometimes a discovery — that your mental model of how the parts relate to the whole was wrong.*

Common causes of reconciliation failure:

- A missing branch (a cost category you forgot)
- Double-counting (a driver that appears in two places)
- Wrong operator (summing something that should be a product)
- Stale data (a leaf value that belongs to a different time period)

Each of these is structurally informative. The reconciliation check is not an error message — it is a MECE check applied to numbers. MECE Studio also catches the most common double-count patterns on its own: a summed term named like a running "total", or two terms with the same label, is flagged before it silently inflates the parent.

Sensitivity analysis: finding the number that matters

Once the tree rolls up correctly, the most powerful thing you can do is ask: **which leaf driver, if it moved, would move the root the most?**

This is sensitivity analysis. The mechanics are simple: move each leaf driver up 10% and down 10%, one at a time, and measure how much the root changes. Then rank the drivers by the size of that swing.

In MECE Studio, this is a one-click operation on any tree that has numeric leaves. The output is a ranked list. For the example above, with the numbers as given, the ranking looks roughly like this:

Driver	±10% swing on Profit	Interpretation
Volume	±100 000 DKK	Largest lever
Input costs	±62 000 DKK	Significant
Operating costs	±38 000 DKK	Moderate
Price	±100 000 DKK	Tied with volume

Wait — Price and Volume tie here because Revenue = Price × Volume, and both are 250 and 4 000 respectively. The tie is arithmetically exact in this symmetric case. In real data, they will diverge.

The point is not the exact ranking. The point is that **the ranking tells you where to spend your investigation time**.

If volume is the largest swing, go find out why volume fell — and whether it can be recovered. If input costs dominate, go understand the cost structure. The sensitivity ranking does not answer those questions, but it tells you which questions to answer first.

> *The sensitivity ranking is a prioritisation tool wearing arithmetic clothes. It does exactly what the impact axis in a prioritisation matrix does — but it computes the impact from the structure of the tree, rather than asking you to estimate it.*

This connects directly to the chapter on **Prioritise**: there, you score branches by impact and ease; here, impact is *derived* rather than *asserted*. Both approaches point at the same thing: where to spend time.

A sensitivity reading in practice

Suppose your sensitivity output shows that volume has a 3× larger swing than price. What does that tell you?

First, it tells you that fighting over a 10% price improvement will move profit less than achieving a 10% volume improvement. If you have a fixed amount of management attention, spend it on volume.

Second, it tells you which assumptions to stress-test. If volume is that sensitive, your profit forecast is fragile to volume assumptions. Before you present numbers to a board, you want the volume estimate to be well-grounded — sourced from pipeline data, customer contracts, or a credible market model. A back-of-the-envelope volume number will undermine the whole tree.

Third, it shapes the conversation. When you present, you can say: "Profit is most sensitive to volume — a 10% shortfall here costs us X DKK, which is three times larger than the same miss on price. So the

conversation we need to have is about the volume trajectory." That is a specific, defensible claim. It comes directly from the tree.

When to stop subdividing

Value-driver trees can grow deep fast. Every leaf can, in principle, be split further — Volume into new customers and retained customers; Input costs into raw materials and energy and logistics. When do you stop?

Stop when the split no longer helps you make a decision or focus an investigation. Ask: *if I had two different numbers for these two children, would I do anything differently?* If the answer is no, the split adds complexity without adding value.

In practice, two to three levels of decomposition is usually sufficient to get from the root question to drivers that someone in the organisation actually controls or can measure. Go deeper only when the sensitivity ranking points you at a node that is still too aggregate to act on.

From numbers to insight

A value-driver tree with a completed sensitivity ranking does something that a spreadsheet cannot: it names the lever. It says, in the language of the problem, which driver is doing the most work — and by how much.

That is the input the next step needs. Once you know which driver matters, you need to assemble the evidence (covered in **Evidence**) and build the argument for what to do about it. The numbers tell you where to look; the synthesis tells you what to say about what you found.

The chapter that follows, **Answer-first**, shows how to take the output of a completed, prioritised, evidence-backed tree and turn it into a communication that works.

Answer-first

You have done the hard work. The question is sharp. The tree is MECE. The high-priority branches are clear. The evidence is in. The numbers, in the chapter **Doing the numbers**, pointed you at the levers.

Now someone is going to ask you what you found.

This chapter is about how to answer them. Specifically, it is about a discipline called **answer-first** — leading with your conclusion, not with the story of how you reached it. It is also about how a well-built issue tree practically writes this communication for you.

The trap: narrating the analysis

The natural instinct, after a thorough investigation, is to tell the story in the order you lived it. Here is the data we collected. Here is what we looked at first. Here is what we ruled out. Here is what we found. Therefore, here is the answer.

This is **bottom-up narration**, and it fails for most professional audiences because:

1. The answer arrives last, after the audience has already spent effort trying to guess where you are going.
2. Senior stakeholders will interrupt before you get there — they will ask "so what's the recommendation?" in the middle of your third slide.
3. If the answer is uncomfortable, building to it slowly signals that you are not fully behind it.
4. It is long. Most of the analysis story is irrelevant to a decision-maker; they need the conclusion and its justification, not the analytical journey.

The answer-first alternative inverts the structure entirely.

The answer-first principle

> *Lead with your governing thought. Then deliver the fewest MECE arguments that prove it. Then the evidence.*

This structure — conclusion first, then reasons, then support — is sometimes called the **pyramid principle** (the McKinsey tradition traces it to Barbara Minto's work). The mental model is a pyramid: the point sits at the top; below it are the supporting arguments; below those are the facts and evidence.

Applied to the "Why are profits falling?" question, the pyramid looks like this:

```
GOVERNING THOUGHT
Profit has fallen 194 000 DKK year-on-year, driven primarily
by a 5% volume decline and a 5% rise in input costs.

SUPPORTING ARGUMENTS
1. Volume fell from 4 200 to 4 000 units (−5%)
2. Input costs rose from 590 000 to 620 000 DKK (+5%)
```

```
3. Price and operating costs were broadly stable

EVIDENCE
- Sales pipeline data showing lost accounts in Q3
- Supplier invoices showing raw material price increases
- P&L comparison confirming stable operating cost base
```

Each level is MECE. The three supporting arguments together account for the full profit gap — no overlap, no gap. The evidence substantiates each argument.

Notice what is missing: the months of analysis, the branches you ruled out, the data cleaning. None of that appears in the communication. It happened; it is why you can stand behind the conclusion; but it is not in the story you tell.

How the tree writes the synthesis

This is the payoff for all the structural work in earlier chapters.

A well-built, prioritised, evidence-backed issue tree already contains everything the pyramid needs — in the right order.

The **root question** becomes the occasion. ("We set out to understand why profits fell.")

The **governing thought** comes from the highest-priority branches plus your hypothesis status. If your Revenue hypothesis was supported ("Revenue is down") and your Costs hypothesis was also supported ("Costs are up"), and sensitivity showed both are significant, your governing thought is: *Profit is being squeezed from both sides — lower revenue and higher costs — with volume and input costs as the primary drivers.*

The **supporting arguments** come from the first-level branches, sorted by priority. The chapter **Prioritise** covered how to score each branch by impact and ease; that score now determines what you say first, second, third. High-priority, confirmed hypotheses become your headline arguments. Low-priority or unconfirmed branches either get a brief note ("Price was broadly stable") or are omitted.

The **evidence** is already on the nodes. In **Evidence** you attached sources, data references, and observations to each node. The synthesis just surfaces them in the right place — under the argument they support.

The MECE check ensures there are no logical gaps in the argument. If the synthesis panel flags an overlap, it means two of your arguments are not independent — you need to restructure before communicating. If it flags a gap, you may be leaving out a factor that your audience will ask about.

None of this requires you to rebuild the communication from scratch. The structure already exists. You are reading it back in the right order.

Action titles: say the so-what, not the topic

There is a small discipline that separates a consulting deck from a school report: every heading states a **finding**, not a **topic**. "Volume declined 5% on Q3 account losses," not "Volume analysis." Barbara Minto

calls this the **action title** — or the **so-what**: the one-sentence conclusion the section actually reached. A reader who skims only your headings should still come away with the whole argument.

An issue tree is built to carry these. Each split has a so-what — the point its branches add up to — and once you write that point down, the tree stops being a set of topics and becomes a set of claims. The root's so-what is your governing thought; each first-level branch's so-what is a supporting argument; and the headings of your eventual deck write themselves.

MECE Studio gives it a home. Any split carries an optional **so-what (insight)** on the inspector's **Logic** tab — a single line, phrased as a finding rather than a label. The synthesis then leads that branch with its so-what instead of the bare node title, and the **root** split's so-what opens the whole synthesis as the governing thought the tree argues for. Fill them in as your branches resolve, and the answer-first narrative the panel produces arrives already written in action titles.

MECE Studio's synthesis panel

The synthesis panel reads your tree and renders it answer-first automatically.

It opens with your **governing answer** — the day-one hypothesis you stated in the **Answer** banner above the canvas — paired with a rolled-up **verdict** computed from the top branches' status: *"3 of 5 top branches supported, 1 refuted — the answer partially holds."* That one line is the pyramid's apex, with the tree's own bookkeeping standing behind it.

Then come the branches, led by the **highest-priority** one and continuing in priority order, descending. For each branch it surfaces:

- The branch's **so-what** action title, where you set one — its headline finding, stated as a conclusion rather than a topic label
- The **hypothesis status** (✓ supported, ✗ refuted, ⊘ parked; open branches carry no mark) so your audience can see what is settled and what is still uncertain
- The **evidence** you attached to that branch — sources, data, observations
- In a value-driver tree, the **numbers**: each node's value and unit, the rolled-up total on a formula parent, and the **most-sensitive driver** — so the answer includes its maths
- Any **MECE flags** — gaps or overlaps the live checker found

The result is a structured narrative you can read top to bottom. It does not write prose for you, but it gives you the skeleton: the right argument, in the right order, with its support visible.

From the panel you can **copy as Markdown** and paste directly into a document, email, or presentation notes. The structure survives the paste — you get headings, bullet points, and evidence inline, ready to edit into prose.

When the deliverable is the answer itself, skip the paste altogether: **Export ▾ → Answer (1-page)** writes a clean, self-contained **HTML memo** — thesis on top, verdict, then the branches in priority order — ready to hand over as-is. It is the actual handoff document, not a canvas screenshot.

Communicating with different formats

The tree synthesis gives you the logical structure. How you package it depends on the format:

Slide deck — One slide per supporting argument. The title of each slide is the argument stated as a conclusion ("Volume declined 5% due to Q3 account losses"), not a topic label ("Volume analysis"). The body of the slide is the evidence. The first slide is the governing thought. This is the "SCR" structure (Situation, Complication, Resolution) or the "action title" convention that consulting decks use — the tree writes it for you.

Email or briefing note — The first paragraph is the governing thought. Each subsequent paragraph covers one supporting argument, in priority order. Evidence is cited inline or attached. This is the format senior stakeholders read on their phones; they stop after the first paragraph unless they want detail.

Meeting or verbal presentation — Start with: "Here is what we found: [governing thought]." Then: "Let me walk you through the three reasons." Then one supporting argument at a time, with evidence. Questions will arrive; the tree structure means you can answer "where does that fit?" precisely — either it is on the tree, or it is not, and you can say which.

Handling supported, refuted, parked, and open hypotheses

Not everything will be settled when you communicate. Some hypotheses stay open because you could not get the data in time. Some are refuted — which is itself a finding worth stating.

The pyramid handles this cleanly:

- **Supported** hypotheses become supporting arguments. State them as facts.
- **Refuted** hypotheses belong in a brief section ("What we ruled out") if your audience is likely to ask, or in an appendix. They are part of the story only if the audience needs to trust the process.
- **Parked** hypotheses — ones you deliberately set aside — show a ⊘ mark in the synthesis. Name them as deferred so no one assumes they were tested and dismissed.
- **Open** hypotheses should be named as uncertainties. "We believe X, but we have not yet confirmed the underlying driver. The risk is Y." Open branches carry no mark — they are simply untested, a signal to you and your audience that work remains.

Hiding open hypotheses is a common failure mode. If you present a conclusion that rests on an unconfirmed assumption, and the audience later discovers this, you lose credibility. Better to name the uncertainty and state what it would take to resolve it.

The loop back to the key question

A synthesis is not just a communication technique. It is also a **quality check** on the analysis.

Read your governing thought back against the key question from the chapter **Start with the question**. Does the governing thought actually answer the question?

"Profits are falling because of volume and input costs" answers "Why are profits falling?" — yes.

"Volume declined in Q3 due to account losses in the enterprise segment" does not answer the original question on its own — it is a supporting argument, not a governing thought.

This check catches a common failure: drifting from the original question as the analysis deepens. You get absorbed in a particularly interesting branch — say, the competitive dynamics behind the volume decline — and end up presenting a detailed answer to the wrong question.

> *The key question is the north star. The governing thought must answer it. If it does not, either reframe the governing thought or go back and check whether the key question has evolved.*

A related check: is the governing thought something your audience can act on? "Profits are falling because of structural industry headwinds" is technically an answer, but it implies helplessness. "Profits are falling primarily because of input cost inflation, which can be partially offset by renegotiating the top three supplier contracts" is actionable. The difference is in the specificity of the supporting structure — which is exactly what the tree provides.

The synthesis as a living document

The synthesis panel in MECE Studio reflects the current state of the tree. As you gather more evidence, mark hypotheses supported or refuted, or adjust priorities, the synthesis updates. This means you can use it iteratively — not just at the end of the process, but throughout.

At each stage-gate in a project, you can read the synthesis and ask: given what we know now, what would we tell the client or the steering committee? This keeps the communication honest. It prevents the common failure of presenting a confident synthesis built on analysis that is actually three weeks out of date.

When you copy the Markdown output, it captures the governing thought, the branches in priority order with their hypothesis status (✓ supported, ✗ refuted, ⊘ parked), the evidence on each, and any outstanding MECE flags — the whole state of the analysis as a structured snapshot. Anyone reading it later sees exactly which hypotheses were settled and which were still open.

What you have, and what it is worth

By the time you reach synthesis, you have:

- A question that is worth answering
- A MECE tree that covers the full problem space
- Hypotheses that direct the analysis toward the most likely answers
- Priorities that tell you which branches to act on first
- Evidence that substantiates the claims
- Numbers (where relevant) that quantify the drivers and rank the levers

What the synthesis does is **make all of that communicable**. The structure that has been invisible to your audience — the tree, the MECE logic, the prioritisation — becomes the backbone of a clear, defensible argument.

Answer-first is not a presentation trick. It is the natural consequence of building the analysis the right way. If the tree is well built, the synthesis is already there. You are reading it out loud.

You have now seen every move in isolation. The next chapter runs them in sequence — one real problem, from a vague worry to a board-ready answer.

A worked example

You have read about the method. You have practised the moves in isolation — sharpening a question, running the MECE test, setting hypotheses, scoring impact × ease, hanging evidence, putting numbers on leaves and reading sensitivity, synthesising answer-first. This chapter puts all of those moves into a single sequence.

No new theory. Just a real problem, worked from the blank canvas to the board-ready paragraph.

The question

Imagine you are a founder. Your SaaS company has been running for four years, profitable for two of them. Over the last three quarters, profits have been falling — not catastrophically, but steadily, and the trend is the wrong direction. You know something is off. You just don't know what.

That feeling — *something is off* — is not a key question. It is a worry. And as the chapter on **Start with the question** explains, you cannot build a useful tree from a worry. The root of your tree needs to be specific, decision-relevant, and answerable.

You sit down and press on it. *What exactly are we trying to answer? What decision hangs on the answer?* The falling profits will eventually require a response — either a cost move, a revenue move, or both. The question is which one, and in what order.

After a few minutes of sharpening, you rename the root node in MECE Studio:

> ***Why have our profits fallen over the last three quarters, and what are the two or three highest-impact moves to reverse the trend?***

That is your key question. It is specific (three quarters, not "recently"), it points toward a decision (moves to reverse it), and it is answerable with data you can actually gather. The whole tree now exists to answer this question and nothing else.

First cut of the tree

Profit is a difference: **Profit = Revenue − Costs**. That is not a choice — it is arithmetic. The chapter on **Ways to decompose** calls this a formula split, and it is almost always the right starting structure for a financial problem because the formula reconciles: once you know revenue and costs, you know profit exactly. There is no overlap and no gap.

You add two sub-issues to the root: **Revenue** and **Costs**. The live MECE dots in MECE Studio turn green immediately — the formula makes the split watertight.

Next, you decompose Revenue. Revenue in a subscription SaaS business is: **Revenue = Active customers × Revenue per customer**. Another formula split. You add both children. The dots stay green.

For Costs, you switch to a segments split — the cost buckets in your business are not connected by a formula, they simply add up. You add four children: **Hosting & infrastructure**, **Headcount**, **Sales &**

marketing, and **Other**. Because this is a segment split, not a formula, MECE Studio nudges you: does "Other" cover anything you haven't named? You check — there is a small software licence cost that doesn't fit neatly. You leave "Other" in place rather than removing it. The CE side is now closed.

The tree at this point looks like this:

Level	Node	Split type
Root	Why have profits fallen?	—
L1	Revenue	Formula (Profit = Rev − Cost)
L1	Costs	Formula (same)
L2	Active customers	Formula (Rev = AC × RPC)
L2	Revenue per customer	Formula (same)
L2	Hosting & infrastructure	Segments
L2	Headcount	Segments
L2	Sales & marketing	Segments
L2	Other	Segments (catch-all)

Eight nodes. Enough structure to think with; not so much that you lose the map.

You will go deeper on the branches that matter. But you do not go deeper yet — as the chapter on **Prioritise the 80/20** warns, decomposing everything before you know where the action is is how you boil the ocean.

Hypotheses

Before you look at a single number, you write down what you already believe. This is hypothesis-driven problem solving: you start from a best guess and use the tree to confirm it or kill it.

Your instinct, sitting in the founder seat: churn has been rising since a major release eight months ago. The release added three new features but broke the onboarding flow for trial users. You suspect a segment of new users never reaches activation, churns before the first renewal, and that is dragging active customers down.

That is your day-one answer, and you give it its official home: in the **Answer** banner above the canvas you type *"Profits are falling because new-user activation broke after the August release."* The whole tree now exists to support or break that sentence — and the synthesis will later open with it, plus a verdict on how it held up.

In MECE Studio you set the **Active customers** node to hypothesis status **Open** and type the hypothesis text: *"Churn has increased since the August release; new-user activation is broken."*

You also have a vaguer suspicion about costs — hosting costs felt higher on the last two invoices. You set **Hosting & infrastructure** to **Open** with a note: *"Infra costs may have crept up with the new feature set."*

Everything else stays at the default (unset). You are not claiming the other branches don't matter — you are just being honest that you don't have a hypothesis about them yet. The open-hypothesis flags are not answers; they are bets you intend to test.

Prioritise

You have eight branches. You cannot work all of them equally. The chapter on **Prioritise the 80/20** says: score impact and ease, then focus where both are high.

You score each L2 node in the impact × ease panel. The scoring is fast — you are working from intuition and a rough sense of the numbers, not from data yet:

Branch	Impact	Ease	Band
Active customers (churn hypothesis)	High	High	**High**
Revenue per customer (pricing)	Medium	Low	Med
Hosting & infrastructure	Low	High	Med
Headcount	Medium	Low	Med
Sales & marketing	Low	Low	**Low**
Other	Low	High	Low

The **Active customers** branch lights up as the obvious first move: if churn is up, recovering even a fraction of it has a large direct effect on revenue, and the data (product analytics, cohort retention) is readily available. You can test the hypothesis cheaply.

The **Hosting & infrastructure** hypothesis is easy to check — one invoice review — but even if it is true, the dollar impact is small relative to a revenue problem. You flag it as a quick parallel check, not the main line of investigation.

You park **Sales & marketing** for now. The tree makes that explicit: a Low-band branch is not ignored, it is deprioritised with a visible reason.

Evidence

You spend two days gathering. You pull cohort data from your analytics platform, look at trial-to-paid conversion rates by signup month, and review the last three infrastructure invoices.

On the Active customers branch:

- Cohort data shows trial-to-paid conversion dropped from 28% to 17% in the cohort that signed up after the August release. You attach this as supporting evidence, strength: **Strong**.
- Month-on-month churn of existing customers is flat. The problem is not retention of paying customers — it is activation of new ones. You attach this as **contradicting** evidence for the specific framing "churn is up" — and update the hypothesis text to be more precise: *"New-user activation broke after the August release; trial-to-paid conversion is down 11 points."* The status stays **Open** — supported on the activation sub-point, but you haven't yet confirmed that this fully explains the revenue decline.
- You find a product ticket from September noting that the onboarding wizard silently fails for users who sign up via Google OAuth. That ticket was closed as "low priority." You attach it as supporting evidence, strength: **Strong**.

At this point you flip **Active customers** to **Supported**. The hypothesis is now effectively confirmed: activation broke, and the data is clean.

On the Hosting & infrastructure branch:

- Invoice review: hosting costs are up £800/month versus a year ago, largely from auto-scaled compute on the new feature. You attach this as supporting evidence, strength: **Medium**.
- But £800/month is roughly £9,600/year. That is real money, but it explains only a small fraction of the profit decline. You set the node to **Supported** and note the finding — but the impact score does not change. This branch is not the main event.

On the Revenue per customer branch:

- You check average contract value (ACV) over the period. It has barely moved — up 2% year-over-year, within noise. You attach a **contradicting** note, strength: **Strong**, and flip the status to **Refuted**. Pricing is not the story.

Three hypotheses entered; one fully supported (activation), one supported but small (hosting), one refuted (pricing). The tree has done its job: it showed you where to look and stopped you spending time on a branch that turned out to be irrelevant.

The numbers

The tree is qualitatively convincing. Before you write a recommendation, you want to quantify it — not to reach a different conclusion, but to sharpen it. The chapter on **Doing the numbers** explains why: numbers tell you which driver dominates, which tells you how big a fix needs to be.

You put illustrative values on the value-driver leaves. All figures are monthly:

Driver	Value	Unit
Active customers (current)	420	customers
Active customers (prior peak)	510	customers
Revenue per customer	180	£/customer/month
Hosting & infrastructure	12,400	£/month
Headcount	38,000	£/month
Sales & marketing	9,200	£/month
Other	2,800	£/month

MECE Studio rolls up:

- **Revenue** = 420 × £180 = **£75,600/month**
- **Total costs** = £12,400 + £38,000 + £9,200 + £2,800 = **£62,400/month**
- **Profit** = £75,600 − £62,400 = **£13,200/month**

You then set the prior-peak scenario — 510 customers at the same revenue per customer — and the roll-up gives:

- **Revenue (peak)** = 510 × £180 = **£91,800/month**
- **Profit (peak)** = £91,800 − £62,400 = **£29,400/month**

The gap is £16,200/month. That is the size of the problem: roughly £194,000 per year in lost profit, almost entirely attributable to the 90-customer shortfall.

Now you run sensitivity on the Active customers leaf at ±10%:

- +10% customers (462 vs 420): Revenue up £7,560/month → Profit up £7,560/month
- −10% customers (378 vs 420): Revenue down £7,560/month → Profit down £7,560/month

You run the same ±10% on Revenue per customer:

- +10% RPC (£198 vs £180): Revenue up £4,200/month → Profit up £4,200/month
- −10% RPC: Profit down £4,200/month

The volume driver is nearly twice as sensitive as the price driver. Even if you raised prices by 10% — a painful move that risks its own churn — the impact is smaller than recovering 42 customers. The numbers confirm what the qualitative evidence already suggested: **fix activation, recover volume, and the profit problem largely solves itself.**

> *Sensitivity tells you which driver deserves the most attention. Here, a 10% swing in active customers moves profit by £7,560/month; the same swing in price moves it by £4,200/month. Fight over the number that matters most.*

You also look at hosting costs: even eliminating the £800/month infra creep entirely moves profit by £9,600/year — meaningful, but less than a single recovered customer-cohort. You note it as a hygiene fix, not a strategic priority.

Answer-first synthesis

The tree is done. You open the synthesis panel in MECE Studio. It opens with the answer from the banner and a rolled-up **verdict** from the branch statuses — activation supported, pricing refuted — and carries the values and the most-sensitive driver into each branch, so the maths travels with the story. You read through the auto-generated summary and tighten the language. Then you take it out in the form the audience needs: **copy as Markdown** to paste into the board deck, or **Export ▾ → Answer (1-page)** for a self-contained one-page memo — thesis, verdict, branches — that attaches to the board pack as-is.

Here is what lands on the page:

Why have profits fallen, and what should we do?

Profits have fallen by roughly £16,200/month (≈£194k/year) since last summer, almost entirely because active customers have dropped from 510 to 420. Revenue per customer is stable and cost categories are broadly in line — the problem is not pricing, not headcount, and not a cost spiral. It is activation. Trial-to-paid conversion dropped 11 percentage points after the August release, tracing to a silent OAuth bug in the onboarding wizard that has been open since September.

The two moves:

1. **Fix the onboarding bug immediately.** Restoring trial-to-paid conversion from 17% toward 28% recovers the lost cohort flow. At current trial volumes, closing half the gap adds roughly 35–40 active customers within 90 days, worth approximately £6,300–7,200/month in profit.
2. **Audit and instrument activation.** The August release introduced the bug; the team didn't see it for weeks because activation was not being tracked by cohort. Add a cohort conversion dashboard to the weekly metrics review so any future regression is caught within a sprint cycle, not a quarter.

Infrastructure cost crept up by £800/month and should be reviewed, but it is a hygiene fix — it does not explain the profit decline and resolving it does not change the strategic picture.

You copy that out of the synthesis panel. It leads with the answer (volume, not price or costs), names the cause (activation bug), quantifies the gap (£194k/year), and gives two concrete moves with enough specificity that an engineering team and a finance team know exactly what to act on. The appendix to the board deck can link to the full tree — every claim is traceable to a branch, a hypothesis status, and a piece of evidence.

More worked examples

This chapter worked one problem end to end. The **Templates** page in MECE Studio carries several more — each a ready-made tree you can open and pull apart the same way:

- **Operating profit** — the value-driver tree behind this chapter, its formula splits already reconciling.
- **Customer churn** — a lifecycle segmentation with an "Other" bucket, carrying hypotheses and evidence.
- **Market entry** — the four-gate test: is the market attractive, can we beat the competition, do we have the capabilities, will it pay off?
- **Acquisition (M&A)** — a "should we buy them?" tree whose synergy branch is a provable formula that has to clear a stated profit goal.
- **Pricing**, **market sizing**, **build vs buy vs partner**, and a **revenue value-driver tree** — each modelling a different way to cut the problem.

Open one and read it the way you would read someone else's working: check each split's type, look for the MECE flags, follow the evidence and the numbers. Reading good trees is one of the fastest ways to build the instinct for cutting your own.

Reflection

Look at what just happened.

You started with a feeling — *the business feels off* — and ended with a specific, auditable answer in roughly three days of focused work. You did not guess. You did not anchor on the first explanation someone suggested. You did not boil the ocean.

The structure did the work.

The formula split told you the problem had to live in revenue or costs, not both. The MECE test forced you to name "Other" in the cost segments, so nothing hid in the gaps. The hypothesis panel stopped you treating

instincts as conclusions. The impact × ease scores kept you from spending two days on a sales-and-marketing hypothesis that would have been a distraction. The sensitivity run told you not to lead with a price increase — which would have been the instinctive move for many operators — because the numbers showed clearly that volume is the bigger lever.

And because the whole analysis lives in a single tree, anyone can audit it. Your CFO can look at the cost branches and verify the cost story. Your head of product can look at the activation evidence and challenge the interpretation. Your investors can trace the £194k figure back to the arithmetic that produced it. The tree is not just a thinking tool — it is a shared object that lets other people think alongside you.

The answer was earned, not guessed. And anyone can check the working.

That is what structured problem solving is for.

One step remains: getting the tree out of the tool and in front of the room — walking it live, printing it, exporting it. That is the final chapter.

Presenting and sharing

The chapter on **Answer-first** was about *what* to say — leading with the conclusion, with the fewest MECE arguments that prove your point — and the worked example showed a full analysis reaching that point. This chapter is about getting the tree out of the tool and in front of the people who need it. A good issue tree is already most of a communication; MECE Studio gives you a few ways to deliver it, and the right one depends on your audience and the moment.

Walking the tree live: presentation mode

When you are in the room — defending your structure in a working session, or taking a team through the logic before the numbers land — you rarely want a finished slide. You want to *walk* the tree, one decomposition at a time, so the audience sees the structure build the way you reasoned it.

Present (from the ⋯ menu) does exactly this. It opens a full-screen, distraction-free view and steps through the tree depth-first: each step shows one question, the branches you split it into, and that split's MECE status. You move with the arrow keys — `→` (or `Space`) forward, `←` back — and leave with `Escape`.

Because it walks the tree top-down, presentation mode naturally enforces the same discipline as answer-first communication: the key question first, then its handful of mutually exclusive branches, then *their* branches. If a slide feels crowded or a branch feels like it overlaps its sibling, the audience will feel it too — which makes presentation mode a useful final check on the structure, not just a delivery tool.

A clean handout: print

Sometimes the deliverable is paper, or a PDF appendix to a deck — something a reader can hold and annotate. **Print…** (from the ⋯ menu) opens a print preview that lays the whole tree out as a clean nested outline: the root question, its type, the MECE summary, and every branch indented beneath its parent — each carrying its hypothesis status, priority band, value, and evidence, so the printed page holds the analysis, not just the labels. The app's own chrome is hidden, so what prints is just the tree. From the browser's print dialog you can send it to a printer or **Save as PDF**.

A printed outline is dense and skimmable in a way a sprawling diagram is not — it is often the better artifact to leave behind after a meeting.

The diagram and the data: export

For everything else, the **Export ▾** menu turns the canvas — or the underlying document — into a file. Pick the format by where it is going:

Format	Best for
PNG	Dropping the diagram straight into a slide or document — a raster image that pastes anywhere.
Copy image	The same rendered tree, straight onto your clipboard — for Slack, a doc, or a slide, no file involved.
SVG	Vector graphics that stay crisp at any size — large-format print, or a slide you will zoom into. The exported file is *sanitised* as it is written, so it can never carry executable content.

Format	Best for
PDF	A single-page, shareable snapshot of the canvas, headed with the tree's title and the date so it reads as a deliverable.
PPTX	A **native, editable** PowerPoint slide: every node is a real text box (carrying its status colour, value, priority, and ME/CE state) and every branch a connector line, so you can rearrange and restyle it in PowerPoint. Titled and dated like the PDF; a very large tree (over ~150 nodes) falls back to an embedded image.
Answer (1-page)	A self-contained **HTML memo** — your governing answer, the verdict, then the branches in priority order. The handoff document itself; see *Answer-first*.
Markdown	The whole analysis as an indented outline — each node with its value, hypothesis status, priority, MECE state, notes, and evidence. Paste it into a doc, a wiki, or an email and the structure travels with it.
CSV (value model)	One row per node — path, label, decomposition type, ME/CE, priority, status, amount, unit, operator — so a value-driver tree opens straight in Excel.
JSON	The raw document, round-tripping with **Open file…** — the format to hand a colleague when you want them to keep *working* on the tree, not just read it.
Copy share link	The whole tree packed into a backend-free `#doc=` URL. Whoever opens it gets the tree as a **new** tree in their own library — nothing is uploaded; the document travels inside the link.

The image and document exporters all render the same canvas, so a PNG, an SVG, and a PDF of the same tree show the same thing in different media. Markdown, CSV, JSON, and the share link go the other way — they carry the *content*, not a picture of it, which is what you want when the tree needs to keep living after the meeting.

One tree, many audiences

The point of all of this is that you build the tree once. The structure you reasoned out — MECE at every split, hypotheses where you have them, evidence and numbers attached — is the single source. Presentation mode, the printed outline, the diagram exports, and the data exports are just different windows onto it. Choose the window for the audience: walk it live with a working group, hand a printed outline or the one-page answer memo to a busy executive, drop a PNG into the board deck, send a colleague the share link, and pass the JSON to the analyst who will take it further.

Appendix A — Glossary

A quick reference to the vocabulary used in this book and in MECE Studio.

Action title (so-what). The one-sentence conclusion a split's branches add up to, stated as a finding rather than a topic ("Volume fell 5% on Q3 losses", not "Volume analysis") — Barbara Minto's term. In MECE Studio you set it on a split's Logic tab; the synthesis leads the branch with it, and the root's action title is the governing thought.

Answer-first. A way of communicating that leads with the conclusion, then supports it — rather than walking the audience through your analysis and revealing the answer at the end. Also called *top-down* or the *pyramid principle*.

Branch. A node in an issue tree together with everything beneath it — a sub-question and its own sub-issues.

Coaching advisory. A light, `info`-level nudge on the *quality* of a tree's thinking — a bare one-word branch, an over-long "laundry list" split, an altitude outlier, a hypothesis still phrased as a question — shown in MECE Studio's Coaching callout. Deliberately separate from the ME / CE checks: coaching never counts toward the MECE health chip.

Collectively exhaustive (CE). A set of parts that, taken together, cover the whole — nothing is left out. The "CE" in MECE.

Deductive split. A split whose children are the steps of an argument chain (premise → premise → conclusion) rather than parts of a whole. MECE does not apply — the steps are meant to build on each other — so the check is whether the chain holds. Contrast *inductive split*.

Dimension. The single axis a split cuts on — by geography, by customer type, by stage. One consistent dimension per level is the backbone of a mutually exclusive split; in MECE Studio you can name it on a split's Logic tab.

Driver. A quantitative input that moves a number you care about. In a value-driver tree, the leaves are drivers (price, volume, cost per unit) and the root is the outcome (profit, revenue).

Evidence. A fact, datapoint, or observation that supports or contradicts a hypothesis. In MECE Studio, evidence is attached to a node with a direction (supporting / contradicting) and a strength.

Exhaustiveness. See *collectively exhaustive*.

Governing answer. The one-sentence answer the whole tree argues for — stated on day one, tested by the branches, and delivered first in an answer-first synthesis. In MECE Studio it lives in the Answer banner above the canvas.

How tree. A prescriptive tree that lays out and compares ways to act ("How do we restore margin?"). A tree should ask one direction — why or how — not both. Contrast *why tree*.

Hypothesis. A candidate answer you intend to test. Hypothesis-driven problem solving starts from a best guess and uses the tree to confirm or kill it, rather than analysing everything in the hope an answer emerges.

Inductive split. A split that cuts a whole into parts of the same kind (regions, cost lines, stages), which MECE tests for overlaps and gaps. The default kind of split. Contrast *deductive split*.

Issue tree. A hierarchical breakdown of a question into sub-questions and sub-issues, so that answering the small questions answers the big one. The central artifact of this book.

Key question. The single, sharp question the whole tree exists to answer. A good key question is specific, decision-relevant, and answerable.

MECE. *Mutually Exclusive, Collectively Exhaustive.* A property of a *split*: the parts don't overlap (ME) and together cover everything (CE). A MECE split is clean to reason about and safe to divide work across.

Mutually exclusive (ME). A set of parts that don't overlap — each item belongs to exactly one part. The "ME" in MECE.

Node. A single box in the tree: a question, sub-issue, or driver.

Ordering principle. The order a split's branches are read in — by *importance* (priority), by *time* (a sequence), or by *structure* (a fixed partition). Barbara Minto's three orders. In MECE Studio a split's ordering principle overrides the global "sort siblings by priority" setting for that split.

Prioritisation. Deciding which branches to work first. In this book, prioritisation is impact × ease: chase the branches that matter and are cheap to test.

Problem brief. A one-page framing of the problem behind the key question — situation, complication, owner, decision-makers, success criteria, scope, and desired outcome (the Problem Identity Card). In MECE Studio, opened from the Answer banner.

Root. The top node of the tree — the key question.

Scaffold. A set of starter sub-issues that a decomposition type suggests (binary → A / not-A, segments → parts + Other). A scaffold is a prompt, not an answer; you rename it.

Sensitivity. How much an outcome moves when one driver changes. A sensitivity analysis ranks the drivers by their effect, so you know which number to fight over.

Split. A parent node together with its immediate children — the unit MECE is judged on. A tree is a stack of splits.

Sub-issue. A child question in the tree; a piece of the question above it.

Synthesis. Reading the tree back as an answer — pulling the findings up the branches into a single, prioritised story.

Value-driver tree. An issue tree whose splits are arithmetic (sum, product, difference), so numbers on the leaves roll up into a number at the root.

Why tree. A diagnostic tree that breaks a problem into its possible causes ("Why have profits fallen?"). A tree should ask one direction — why or how — not both. Contrast *how tree*.

Appendix B — Keyboard reference

MECE Studio is built for flow — you can construct, navigate, and edit a whole tree without leaving the keyboard. These shortcuts work on the canvas; they're ignored while you're typing in an inspector field, so they never hijack your input.

Building and editing

Keys	Action
`Tab`	Add a child to the selected node and start editing it
`Shift + Enter`	Add a sibling to the selected node and start editing it
`Enter` or `F2`	Edit the selected node's label
Double-click	Edit a node's label inline
`Enter` (while editing)	Commit the label
`Escape` (while editing)	Cancel the edit, keep the old label
`Delete` or `Backspace`	Remove the selected node(s) and their subtrees
`P`	Bump the selected node's priority (none → low → medium → high)

Navigating

Keys	Action
`↑` / `↓`	Move selection between siblings
`←`	Select the parent
`→`	Select the first child

Selecting more than one node

Keys	Action
`⌘/Ctrl` + click or `Shift` + click	Add a node to the selection
`Shift` + drag on empty canvas	Rubber-band (box) select everything inside

With several nodes selected, a floating action bar sets their status or priority, or deletes them — one undoable step. A plain drag still pans.

History

Keys	Action
`Ctrl/⌘ + Z`	Undo
`Ctrl/⌘ + Y` or `Ctrl/⌘ + Shift + Z`	Redo

Finding

Keys	Action
Type in the Find box	Ring every node whose label matches
`Enter` (in the Find box)	Zoom to the matching nodes

Elsewhere in the app

Keys	Where	Action
`⌘K` / `Ctrl+K`	Start page	Jump to All trees and focus the library search
`⌘/Ctrl + Enter`	Quick add issues	Add the typed issues
`→` / `Space`, `←`	Presentation	Next / previous step
`Escape`	Dialogs, presentation	Close / exit

Help

Keys	Action
`?`	Open the keyboard-shortcuts overlay (this list, inside the app)

A keyboard-first workflow

To build a tree quickly, with your hands mostly on the keyboard:

1. Select the root and press `Enter` to name your key question.
2. Press `Tab` to add a sub-issue and type its label; `Enter` commits.
3. `Shift + Enter` adds the next sibling — so a whole level is `Tab` once, then `Shift + Enter` for each brother branch.
4. Walk the tree with the arrows — `↑`/`↓` between siblings, `←` to the parent, `→` into a branch — and `Tab` wherever the structure needs to go deeper.
5. Use `Ctrl/⌘ + Z` freely; every change is undoable.

The same labels and structure you build this way are what feed the MECE checks, the synthesis, and every export.

Appendix C — Further reading

This book is a practical, tool-first introduction. If you want to go deeper into the thinking behind issue trees and structured problem solving, these are the standard references. They are listed for study; the authors and publishers are not affiliated with MECE Studio, and their work is their own.

On structured problem solving

- **Barbara Minto — *The Pyramid Principle*.** The classic on answer-first communication: how to order ideas so a conclusion lands first and its support follows in a MECE structure. If you read one book on synthesis, read this.
- **Ethan Rasiel — *The McKinsey Way*** and **Rasiel & Friga — *The McKinsey Mind*.** Accessible accounts of hypothesis-driven problem solving, issue trees, the "80/20" instinct, and how analysis is structured and communicated in practice.
- **Charles Conn & Robert McLean — *Bulletproof Problem Solving*.** A modern, worked treatment of the seven-step problem-solving process, with logic trees (issue trees and value-driver trees) at its centre and many case studies.

On thinking and decisions

- **Daniel Kahneman — *Thinking, Fast and Slow*.** Why our unaided judgement misleads us — the case for externalising reasoning into an explicit structure in the first place.
- **Richard Heuer — *Psychology of Intelligence Analysis*.** On competing hypotheses and weighing evidence; a rigorous complement to the evidence and hypothesis chapters of this book.

On the methods' lineage

The "issue tree", "MECE", and hypothesis-driven approach are most associated with **McKinsey & Company** and the broader management-consulting tradition, and have been described in many of the works above. MECE Studio implements these ideas as a generic, free tool; it is not a McKinsey product and is not endorsed by McKinsey & Company.

The tool

- **MECE Studio** — the free, local-first app this book is written alongside: https://mece-studio.struktureretsundfornuft.dk. The in-app **User Guide** documents every feature and shortcut; this book teaches the method.

Appendix D — Working in MECE Studio

The body of this book is about the *method*. This appendix is a reference to the tool: the canvas, the editing model, the MECE review surfaces, the library, the settings, and the app itself — so you can work quickly and keep your trees safe.

Capturing a decomposition fast

When you already know the branches — in your head, or on a whiteboard — you don't need to add them one node at a time. **Quick add issues…** (from the ⋯ menu) opens a box where you type one issue per line; on confirm, every line becomes a child of the selected node (or of the root, if nothing is selected), all in a single undoable step. It builds levels too: **indent a line** (Tab, spaces, or a bullet) and it nests as a sub-issue, so a whole multi-level outline drops in at once. `Ctrl/⌘ + Enter` adds them without the mouse.

Starting from material you already have

You will often have the makings of a tree already — an agenda, a bulleted brief, a tree someone sent you. **Import outline…** (from the ⋯ menu) turns it into a tree:

- **A Markdown outline.** Paste headings and/or bullet lists; nesting comes from indentation. The first heading or line becomes the root question, and everything else nests beneath it. It is a structural import — you get the hierarchy and labels, ready to refine.
- **An OPML outline.** The export format of most outliners and mind-mappers (MindManager, OmniOutliner, Workflowy, Dynalist, …) — nested `<outline>`s become the tree.
- **A tree's JSON.** Paste the JSON a colleague exported and it opens as a fully restored tree (the same format **Export ▾ → JSON** produces).

The format is auto-detected — you just paste.

Either way, the import opens as a **new** entry in your library, so it never disturbs the tree you are already working on.

The canvas

The tree lays itself out. You never drag nodes into position; MECE Studio runs an **auto-layout** (a left-to-right tree) and **re-fits the view** whenever the tree changes, so the structure stays readable as it grows. Each node is also sized to its own content, so a richer node — one showing a value, evidence badges, and ME / CE dots — is laid out taller than a bare label and never overruns its neighbours; simple trees lay out exactly as before. What you control is the *structure*, and the canvas gives you a few ways to manage a large one:

- **Re-parent by dragging.** Drag any node onto another to move it — and its whole subtree — under that node. While you drag, the valid drop target is ringed, so the result is predictable; an invalid drop (onto the node's own subtree, or the root) simply snaps back.
- **Collapse and expand.** Any node with children gets a toggle: collapse it to hide its subtree and focus on one branch (the node shows a count of what's hidden). **Collapse all** / **Expand all** fold or unfold the whole tree at once.

- **Find.** The search box rings every node whose label matches as you type and shows a match count; press `Enter` to zoom to the matches — invaluable in a big tree.
- **Minimap.** A minimap keeps you oriented in a big tree: one dot per node, coloured by state — amber for a flagged split, blue for a high-priority branch.
- **Select several nodes at once.** `⌘/Ctrl`- or `Shift`-click nodes, or `Shift`-drag a box on empty canvas, and a floating action bar sets their status or priority — or deletes them — in one undoable step.

The canvas also *shows* the analysis at a glance, without opening the inspector: each decomposed node carries **ME / CE status indicators** (is this split mutually exclusive? collectively exhaustive?), a **coloured status edge** for its hypothesis state, and **evidence-count badges** for supporting and contradicting items. Edges out of a flagged split carry a subtle always-on amber tint; when the review dock is open, the canvas goes further and dims the clean splits entirely, so the splits that need attention stand out. (To assistive tech the canvas is a real *tree* — every node announces its depth, expanded state, and selection to a screen reader.)

The first time you face a bare root node, a dismissible **coach tip** on the canvas points at the two moves that start a tree: `Tab` to add a branch, then the **Logic** tab to choose how it splits.

Editing nodes

Most editing happens on the canvas or in the inspector on the right:

Action	How
Rename a node	Double-click it, or select it and press `Enter` / `F2`
Add a child	`Tab` on the selected node, or click the `+` on its child edge (see Appendix B)
Add a sibling	`Shift + Enter` on the selected node
Delete a node and its subtree	Select it and press `Delete` / `Backspace`
Duplicate a subtree	From the inspector — copies the node and its whole subtree (fresh ids) as a sibling
Reorder siblings	Move a node up or down among its siblings, from the inspector
Add notes	The inspector's notes field, for rationale, assumptions, or sources; a node with notes shows a marker, and notes flow into the Markdown export

The **inspector is tabbed** — *Issue · Logic · Evidence · Value* — so you see one facet of the selected node at a time; the **Logic** tab opens automatically when the node's split needs a MECE review. Every change is undoable: `Ctrl/⌘ + Z` / `Ctrl/⌘ + Y`. Press `?` at any time for the keyboard-shortcuts overlay.

Reviewing MECE

MECE checking is the point of the tool, and it surfaces in three places:

- **In the inspector.** A flagged split shows a plain-language explanation of *why* — siblings that may overlap (ME), or children that may not cover the parent (CE).
- **The review dock.** A **MECE health** chip in the header reads ✓ *MECE clean* or ⚠ *N to review* and opens a triage dock: flags grouped into **Overlaps** and **Gaps**, ranked by branch priority, each row with its plain-

language reason, a one-click **Locate** that centres the node on the canvas, a **Review logic** → jump to the inspector, and — for gaps — a concrete remedy (add an "Other" bucket, or a sub-issue).
- **Needs review.** The Start page's **Needs review** section triages your whole library down to the trees that have at least one flagged split.

The Logic tab and coaching

Everything about *how a node splits* lives on the inspector's **Logic** tab, in one place:

- The **decomposition type** (binary / segments / process / formula / framework / freeform) and its scaffold.
- The **dimension** the split cuts on — its single axis, with one-click common ones (customer / geography / product / time / stage).
- **Inductive or deductive.** Mark a split a deductive argument chain and the MECE overlap-check steps aside — an argument builds on itself rather than partitioning (see *MECE: no overlaps, no gaps*). Inductive is the default.
- The **ordering principle** — *importance* / *time* / *structure* — which fixes how this split's branches are ordered, overriding the global priority sort (see *Prioritise the 80/20*).
- The **so-what (insight)** — the one-line action title the branches add up to, which the synthesis leads the branch with (see *Answer-first*).

Below the strict ME / CE explanation, the Logic tab also carries a **Coaching** callout — lighter, `info`-level nudges from the problem-structuring canon (a bare one-word branch, a laundry list of more than ~7 branches, an altitude outlier, a hypothesis still phrased as a question, key-question quality, and why/how consistency). Coaching is deliberately kept **out** of the MECE health chip, so the chip keeps meaning mutual-exclusivity and collective-exhaustiveness alone.

Framing: the Answer banner and Problem brief

Above the canvas sits the **Answer banner** — the one-line governing answer the whole tree argues for. State your day-one hypothesis there and the synthesis opens with it and a rolled-up verdict. Its **Problem brief** button opens the **Problem Identity Card** — *Situation · Complication · Owner · Decision-makers · Success criteria · In scope · Out of scope · Desired outcome* — the fuller framing behind the key question (see *Start with the question*). The brief is also where you tag the tree a **why** (diagnostic) or **how** (prescriptive) tree; a badge in the header then shows which, and the coaching keeps the tree pointed one way.

The library and the Start page

MECE Studio opens on the **Start page** — a workspace shell with a sidebar (**Start, All trees, Recent, Templates, Needs review**, and **Learn MECE**, a short in-app primer with links to the user guide and this book). From here you **start a new tree** from a key-question box — picking how the first split should cut, or starting blank — or open an existing one. Every saved tree appears as a **card** showing a mini preview and a live MECE pill that reads the same status the canvas does; from a card you can **rename, duplicate, or delete** the tree. The library **search** (`⌘K` / `Ctrl+K`) matches node labels and notes across every tree, not just titles — so it finds the tree that *contains* the thought you're looking for. The **Templates** page surfaces every

decomposition style, named framework, and example tree as a one-click card — plus **your own templates**: **⋯ → Save as template…** banks any tree's structure (labels, splits, dimensions; values, evidence, and status stripped) as a clean, reusable starting point.

Open several trees at once and each gets a **tab** above the canvas — with a per-tab **MECE health dot** (green clean / amber to-review / grey undecomposed), so you can see which open trees still need work. The open set survives a reload.

If you used an earlier, single-tree version of MECE Studio, your tree is folded into the library automatically the first time you open the new version — nothing is lost.

Settings

A **⚙ Settings** panel (saved on your device) carries a few preferences, all defaulting to today's behaviour:

- **Sort siblings by priority** — lay branches out highest-impact first, instead of creation order (opt-in).
- **Stricter overlap detection** — also flag shorter shared words between siblings when looking for overlap (opt-in).
- **Formula tolerance** — how closely a value-driver split must reconcile to read as MECE (default 0.5%).

The app itself

- **Everything autosaves** to your browser's local storage, so your trees are there when you return; the file open/save above is layered on top for real `.json` files.
- **Installable and offline.** MECE Studio is a PWA — install it from your browser and it runs offline, like a native app.
- **Self-updating.** When a new version is deployed, a running tab shows a non-intrusive *"A new version is available — Refresh now"* prompt rather than reloading under you; **About → Check for updates** forces a check on demand. The **About** dialog also links the user guide, this book (PDF and EPUB), third-party notices, and the source code.
- The **editor header** is grouped into labelled clusters — brand and title, the MECE health chip, undo/redo, Synthesis, the **Export ▾** menu, Settings, shortcuts, and an **⋯ overflow** menu — so the actions stay findable as the app grows. Confirmations and renames use the app's own dialogs, never a browser popup.
- **On a phone**, the workspace adapts: the header collapses into the **⋯** menu and the inspector and review dock become a **bottom sheet** that rises when you select a node, so the canvas keeps the full width.

AI assist, without an API key

MECE Studio has no backend and asks for no API key, but it still helps you use an LLM. Two actions copy a ready-made prompt with your tree embedded, to paste into Claude or ChatGPT:

- **Critique this tree's MECE** — from the Synthesis panel.
- **Suggest a MECE split for this node** — from the inspector. The prompt asks the LLM for a paste-ready Markdown outline, and a **"paste the AI's split back"** box beneath it grafts those sub-issues straight under the node — closing the loop with no key and no backend.

You stay in control of what you send and where; the tool just writes the prompt.